DATA STRUCTURES LABORATORY

OPEN-ENDED PROJECT REPORT

RAILWAY RESERVATION SYSTEM

FIFO Linked-List Queues with Automatic Seat Allocation and Waiting-List Promotion

Implemented in C with a GTK3 Graphical Interface (Master V4)

GROUP 1

Sl. No.	Name of Student	Roll No.
1	ANIL VK	10
2	AMAN AP	8
3	MIDHUN M	25
4	WILLIAM WILSON	36

Department of Computer Hardware Engineering

Name of Institution: Government Polytechnic College Palakkad

Academic Year 2026

Abstract

Ticket reservation is one of the most natural real-world applications of the queue data structure: passengers are served in the order in which they arrive, and when seats run out, later arrivals wait in line until a cancellation frees a seat. This project implements a complete railway reservation system for a single train, the Kerala Express (No. 12625, Thiruvananthapuram to New Delhi), built around two hand-written FIFO queues implemented as singly linked lists in C: one for confirmed passengers and one for the waiting list.

The system allocates seats automatically across four classes (Sleeper 20, AC 3-Tier 10, AC 2-Tier 5, AC First 3), supports a waiting list of up to 10 passengers, and, when a confirmed ticket is cancelled, promotes the earliest waiting passenger of the same class into the vacated seat. Beyond the core data structure, the project adds input validation, PNR generation, search by Ticket ID or PNR, class and status filters, an e-ticket dialog that can be saved to a file, a live activity log, dashboard statistics and a light/dark theme, all in a responsive GTK3 interface of roughly 2,700 lines of C.

The report explains the data-structure design, the algorithms and their time complexity, the software architecture, the testing carried out, and the limitations and possible extensions. The complete source code is included in Chapter 12.

Keywords: Queue, singly linked list, FIFO, waiting list, dynamic memory, GTK3, C language, reservation system.

Project at a Glance

Item	Details
Project title	Railway Reservation System (Master V4)
Core data structure	Two FIFO queues implemented as singly linked lists (confirmed and waiting)
Train modelled	Kerala Express, No. 12625, Thiruvananthapuram to New Delhi
Seat capacity	38 confirmed (SL 20, 3A 10, 2A 5, 1A 3) plus 10 on the waiting list
Key operations	Book, cancel, same-class FIFO promotion, search by Ticket ID or PNR, filter, e-ticket
Language and toolkit	C (GCC) with GTK3 and GLib
Program size	2,688 lines in a single source file
Team	Group 1: Anil VK (10), Aman AP (8), Midhun M (25), William Wilson (36)

Table of Contents

Abstract 2
Project at a Glance 2
Table of Contents 3
1. Introduction 5
1.1 Why this project is open-ended 5
2. Problem Statement and Objectives 5
2.1 Problem statement 5
2.2 Objectives 5
3. Technology Stack and System Requirements 6
4. Data Structure Design 6
4.1 The Passenger node 6
4.2 The Queue structure 7
4.3 Capacity model 7
5. Algorithms and Complexity Analysis 8
5.1 Queue operations 8
5.2 Seat allocation: make_seat() 8
5.3 Booking algorithm 9
5.4 Cancellation and promotion algorithm 9
5.5 Complexity summary 9
6. System Architecture 10
6.1 Module map 10
7. Implementation Details 11
7.1 Input validation 11
7.2 Memory safety 11
7.3 PNR and Ticket ID generation 12
7.4 Searching and filtering 12
7.5 E-ticket 12
7.6 Logging and notifications 12
8. User Interface 12
9. Build and Execution 13
9.1 Compilation 13
9.2 Running the application 13
10. Testing 13
11. Output and Results 15
11.1 Summary of results 15
11.2 Output screenshots 15

12. Source Code: Complete Program Listing 18
12.1 Header, macros, data structures and global state (lines 1 – 160) 18
12.2 Queue engine (lines 161 – 268) 20
12.3 Utility and validation functions (lines 269 – 476) 21
12.4 Logging (lines 477 – 515) 24
12.5 UI helper functions (lines 516 – 625) 24
12.6 Dashboard, filters and table refresh (lines 626 – 993) 26
12.7 E-ticket generation, saving and dialog (lines 994 – 1200) 30
12.8 Event handlers: booking, cancellation, search, reset (lines 1201 – 1635) 33
12.9 Notifications, clock, theme and navigation (lines 1636 – 1789) 39
12.10 TreeView setup and CSS styling (lines 1790 – 1953) 41
12.11 Page builders: dashboard, booking, passengers, activity, settings (lines 1954 – 2574) 43
12.12 Main window and main() (lines 2575 – 2688) 52
13. Advantages, Limitations and Future Scope 54
13.1 Advantages 54
13.2 Limitations 54
13.3 Future scope 54
14. Team Members and Contribution 54
15. Conclusion 55
References 55

1. Introduction

A queue is a linear data structure that follows the First-In-First-Out (FIFO) principle. Anything that is fair to the person who arrived first, such as a ticket counter, a print queue or a call centre, is a queue at heart. Railway reservation adds a twist that makes it a rich subject for a data structures laboratory: there are two related queues (confirmed and waiting), elements must sometimes be removed from the middle rather than only from the front, and one queue feeds the other when a seat is released.

This project takes that idea from a console exercise to a complete desktop application. Rather than relying on arrays or library containers, all passenger records are stored in dynamically allocated nodes linked by pointers, so every insertion, removal and promotion is visible in the code as pointer manipulation. The graphical interface is built with GTK3 so that the data structure can be observed working in real time through tables, statistics and an activity log.

1.1 Why this project is open-ended

The brief for an open-ended project does not prescribe a single output. It expects the team to choose a real problem, decide which data structures fit, justify those choices, and extend the solution as far as time allows. In this project the following decisions were ours to make:

- **Choice of structure:** a linked-list queue was chosen over an array queue so that removal from the middle (cancellation) does not require shifting elements and capacity is enforced by policy rather than by a fixed array.
- **Business rules:** per-class seat capacities, a bounded waiting list, and promotion restricted to the same class as the cancelled seat.
- **Robustness:** validation of name, phone, e-mail and date, safe string copying, and checks on every memory allocation.
- **User experience:** a full GUI with dashboard, booking form, passenger management, activity log and settings, rather than a text menu.
- **Extensions:** search by PNR, filters, a printable/saveable e-ticket, dark mode and confirmation dialogs.

2. Problem Statement and Objectives

2.1 Problem statement

To design and implement a railway reservation system that books tickets for a fixed-capacity train, places passengers on a FIFO waiting list when a class is full, and automatically promotes waiting passengers when confirmed tickets are cancelled, while keeping the passenger records consistent and the interface easy to use.

2.2 Objectives

1. Implement a generic FIFO queue using a singly linked list with front and rear pointers.

2. Use two queue instances, confirmed and waiting, to model the reservation process.
3. Allocate seats automatically and never issue the same seat twice within a class.
4. Promote the first waiting passenger of the same class when a confirmed ticket is cancelled.
5. Validate all user input before any record is created.
6. Allow passengers to be found by Ticket ID or PNR and filtered by class and status.
7. Present live statistics, an activity log and a printable e-ticket.
8. Release all dynamically allocated memory on cancellation, reset and exit.

3. Technology Stack and System Requirements

Item	Details
Language	C (compiled with -Wall -Wextra -Wpedantic, so the build produces no warnings)
GUI toolkit	GTK 3 (GtkHeaderBar, GtkStack, GtkStackSidebar, GtkTreeView, GtkListBox, GtkInfoBar, CSS styling)
Support library	GLib (timers, memory helpers, text buffers)
Build tool	GCC with pkg-config
Operating system	Ubuntu / Debian-based Linux with a graphical session
Packages	gcc, pkg-config, libgtk-3-dev
Program size	About 2,700 lines in a single source file, railway_reservation_v4_responsive_no_top_bar.c

4. Data Structure Design

4.1 The Passenger node

Every reservation is one heap-allocated node. Besides the passenger details it carries the Ticket ID, PNR, seat, waiting position and a next pointer that links it to the following node in whichever queue it currently belongs to.

```
typedef struct Passenger {
    int ticket_id;
    char pnr[PNR_LEN];
    char name[NAME_LEN];
    int age;
    char gender[GENDER_LEN];
    char phone[PHONE_LEN];
    char email[EMAIL_LEN];
    char travel_class[CLASS_LEN];
    char from[FROM_LEN];
    char to[TO_LEN];
    char date[DATE_LEN];
    char berth[BERTH_LEN];
    char seat[SEAT_LEN];
    int waiting_position;
```

```
    struct Passenger *next;
} Passenger;
```

Field	Type	Purpose
ticket_id	int	Unique sequential ID starting at 1001
pnr	char[20]	10-digit Passenger Name Record starting from 4817263951
name, age, gender	char[48], int, char[12]	Passenger identity
phone, email	char[20], char[80]	Contact details (validated)
travel_class	char[8]	SL, 3A, 2A or 1A
from, to, date	char[32], char[32], char[20]	Route and journey date (DD-MM-YYYY)
berth, seat	char[24], char[16]	Berth preference and allocated seat such as 3A-04 (empty when waiting)
waiting_position	int	Position on the waiting list, recomputed on every refresh
next	struct Passenger *	Link to the next node in the queue

4.2 The Queue structure

```
typedef struct {
    Passenger *front;
    Passenger *rear;
    int size;
} Queue;
```

A Queue holds only a front pointer, a rear pointer and a size counter. Keeping a rear pointer is what makes enqueue O(1); without it every insertion would need to walk the entire list.

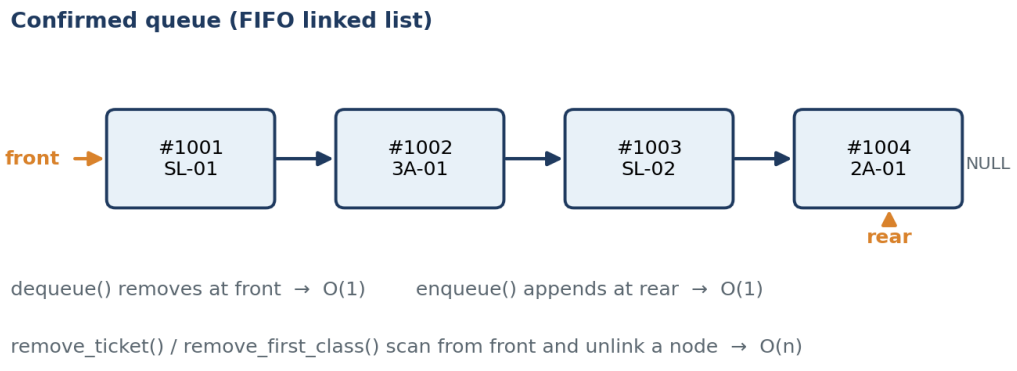


Figure 1: The confirmed queue as a singly linked list with front and rear pointers

4.3 Capacity model

Class	Capacity	Meaning
SL	20	Sleeper
3A	10	AC 3 Tier
2A	5	AC 2 Tier

Class	Capacity	Meaning
1A	3	AC First
Total confirmed	38	MAX_CONFIRMED (sum of class capacities)
Waiting list	10	MAX_WAITING (shared across classes)

A booking is confirmed only when both conditions hold: the train has fewer than 38 confirmed passengers and the requested class has not reached its own capacity. Otherwise the passenger joins the waiting queue, and if that too is full the booking is rejected.

5. Algorithms and Complexity Analysis

5.1 Queue operations

enqueue(q, p): insert at rear

```
set p->next = NULL
if q->rear exists:  q->rear->next = p
else:                q->front = p      (queue was empty)
q->rear = p;  q->size++
```

dequeue(q): remove from front

```
if queue is empty: return NULL
p = q->front;  q->front = p->next
if q->front is NULL:  q->rear = NULL
p->next = NULL;  q->size--;  return p
```

remove_ticket(q, id) and remove_first_class(q, class): unlink from the middle

Both functions walk the list keeping a prev pointer. When the target is found, prev->next is set to cur->next (or front is advanced if the target is the first node). If the target was the last node, rear is moved back to prev. This rear-pointer fix-up is the classic bug in linked-list queues and is handled explicitly.

```
prev = NULL;  cur = q->front
while cur:
    if match(cur):
        if prev: prev->next = cur->next
        else:    q->front   = cur->next
        if q->rear == cur: q->rear = prev
        cur->next = NULL;  q->size--;  return cur
    prev = cur;  cur = cur->next
return NULL
```

5.2 Seat allocation: make_seat()

Seats are numbered per class (for example SL-01 to SL-20). Because passengers can cancel out of order, the next free seat is not simply "count + 1". The function builds a boolean used[] array by scanning the confirmed queue for seats of the requested class, then returns the lowest-numbered seat that is not marked used. A cancelled seat is therefore reused, and two passengers can never hold the same seat.

5.3 Booking algorithm

1. Read all form fields and trim leading and trailing whitespace.
2. Validate selections, name, phone (10 to 15 digits), e-mail and date (DD-MM-YYYY with leap-year checking).
3. Reject the booking if both the confirmed and waiting queues are full.
4. Allocate a node with calloc, assign the next Ticket ID and PNR, and fill in the record.
5. If the train and the class both have room: allocate a seat, enqueue into confirmed.
6. Otherwise, if the waiting list has room: set the waiting position and enqueue into waiting; if not, free the node and report failure.
7. Log the event, refresh the tables and statistics, and show the e-ticket dialog.

5.4 Cancellation and promotion algorithm

Cancellation is the heart of the project because it is where the two queues interact. If the cancelled ticket holds a seat, it is removed from the confirmed queue, its seat and class are remembered, and remove_first_class() searches the waiting queue for the earliest passenger of that same class. If one exists, that passenger inherits the seat and is enqueued into the confirmed queue. If none exists, the seat is simply released. If the cancelled ticket was on the waiting list, it is removed and the passengers behind it move up one position.

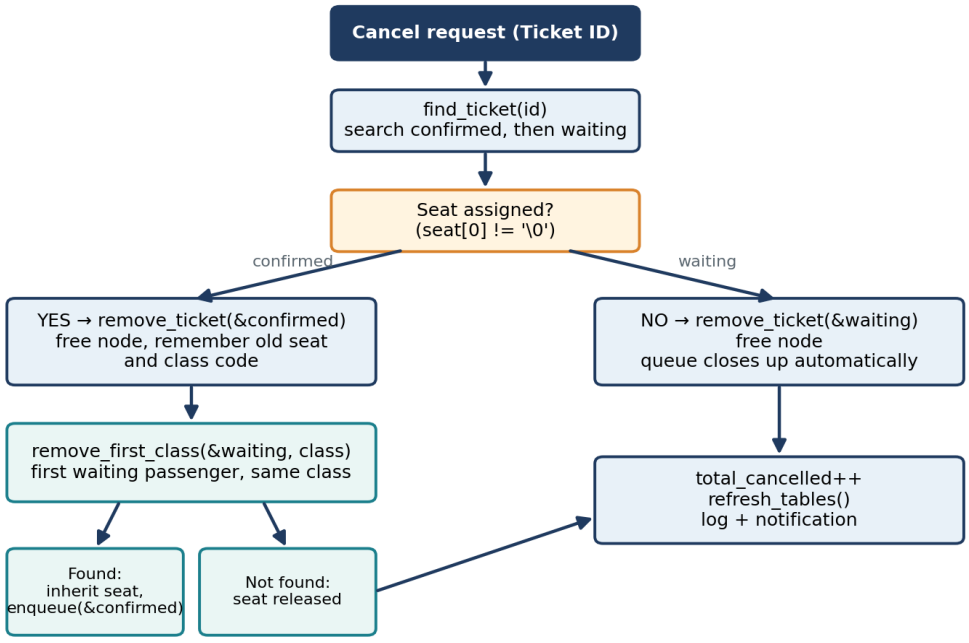


Figure 2: Cancellation and same-class FIFO promotion

5.5 Complexity summary

Operation	Time	Remark
`queue_init, enqueue, dequeue`	O(1)	Pointer updates only, thanks to the rear pointer

Operation	Time	Remark
`find_ticket, find_pnr`	O(n)	Linear scan of confirmed and then waiting (n ≤ 48)
`remove_ticket, remove_first_class`	O(n)	Scan with prev pointer, then O(1) unlink
`confirmed_class_count`	O(n)	Counts one class in the confirmed queue
`make_seat`	O(n + c)	n = confirmed size, c = class capacity (≤ 20)
`free_queue`	O(n)	Dequeues and frees every node
Space	O(n)	One node per passenger, at most 48 nodes

Because the train is bounded to 48 live records, O(n) scans are effectively constant-time and the interface responds instantly. The design would still scale, since the only linear operations are those that inherently need to look at each record.

6. System Architecture

The program is organised in three layers inside a single source file. The lowest layer contains only C and the two queues, and could be reused in a console program without change. The middle layer implements business rules as GTK signal callbacks. The top layer builds the five pages of the interface.

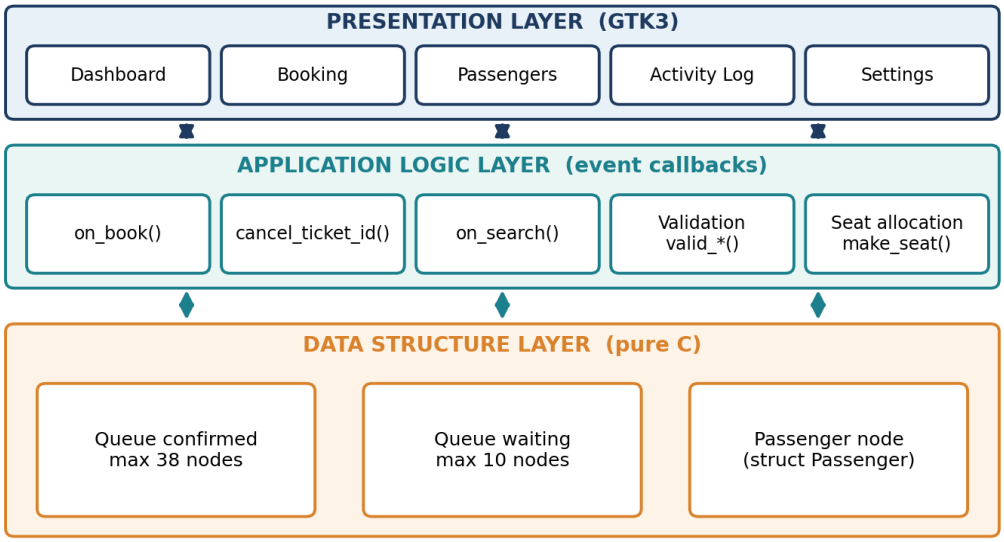


Figure 3: Three-layer architecture of the application

6.1 Module map

Module	Source lines	Main functions
Constants, structs, globals	1 – 160	Passenger, Queue, capacity macros, widget pointers

Module	Source lines	Main functions
Queue engine	161 – 268	queue_init, enqueue, dequeue, remove_ticket, remove_first_class, free_queue
Utilities & validation	269 – 476	copy_text, trim_spaces, make_seat, make_pnr, valid_date/phone/email
Logging	477 – 515	append_activity, log_msg (timestamped, printf-style)
UI helpers	516 – 625	make_label, make_card, make_class_card, make_badge
Dashboard & tables	626 – 993	refresh_dashboard, refresh_tables, filters, recent bookings
E-ticket	994 – 1200	ticket_text, save_ticket_to_file, show_ticket_dialog
Event handlers	1201 – 1635	on_book, cancel_ticket_id, on_search, on_reset, on_new_journey
Notifications, clock, theme	1636 – 1789	show_notification, update_clock, apply_theme, go_to_page
Styling & widgets	1790 – 1953	load_css (light/dark), TreeView setup, route card
Pages	1954 – 2574	Dashboard, Booking, Passengers, Activity, Settings
Window & main	2575 – 2688	build_main_window, main

7. Implementation Details

7.1 Input validation

No record is created until every field passes validation. Phone numbers must contain 10 to 15 digits and may include +, -, spaces and brackets. E-mail addresses need an @ that precedes a final dot with text after it. Dates must follow DD-MM-YYYY, from 2024 onwards, with correct month lengths and leap years (divisible by 400, or by 4 but not by 100). Ticket IDs are parsed with strtol and errno so that overflow and trailing junk are rejected.

7.2 Memory safety

- **Passenger nodes are created with calloc, so every field starts zeroed, and the result is checked for NULL.**
- **Text is copied with snprintf through copy_text(), preventing buffer overflows on fixed-size fields.**
- **A removed node is unlinked (next = NULL) before being freed, so no dangling pointer stays in a queue.**
- **free_queue() runs on reset and on exit, and GTK strings returned by combo boxes are released with g_free().**

7.3 PNR and Ticket ID generation

Ticket IDs start at 1001 and increase by one for each booking. PNRs are generated from a 64-bit counter beginning at 4817263951 and formatted as a zero-padded 10-digit string, so they are unique within a run. Both counters return to their starting values when the system is reset.

7.4 Searching and filtering

The search box accepts either a Ticket ID or a PNR. The input is first tried as a number and looked up with find_ticket(); if that fails it is looked up as a PNR with find_pnr(). A hit selects the passenger, fills the details panel and pre-fills the cancel box. The passenger page can also be filtered by class (All, SL, 3A, 2A, 1A) and by status (All, Confirmed, Waiting), applied when the tables are rebuilt.

7.5 E-ticket

After each booking an e-ticket dialog opens showing the PNR, Ticket ID, seat or waiting position, class, route and date. The passenger can copy the PNR to the clipboard or save the ticket as a text file.

7.6 Logging and notifications

log_msg() accepts a printf-style format, prefixes a timestamp and writes to both the Activity Log page and the dashboard timeline. Short-lived banners (GtkInfoBar) tell the user the outcome of each action: confirmed, waiting, promoted, cancelled or error.

8. User Interface

Page	Function
Dashboard	Live counts of confirmed, waiting, available and cancelled tickets; total bookings and promotions; per-class availability cards with progress bars; recent bookings; live activity timeline; route card.
Book Ticket	Form for name, age, gender, phone, e-mail, class, berth preference and date, with a live availability indicator for the chosen class.
Passengers	Confirmed and waiting tables, search by Ticket ID or PNR, class/status filters, selected-passenger details and cancellation.
Activity Log	Full timestamped history with clear, New Journey and Reset System actions.
Settings	Train profile, dark mode switch and confirmation-dialog preference.

The layout is responsive: the sidebar is fixed-width while the content area expands and scrolls vertically, so the application remains usable when the window is resized. The header bar shows the train name and number, a "SYSTEM ONLINE" indicator and a live clock updated every second.

9. Build and Execution

9.1 Compilation

Place the source file in the Downloads folder, open a terminal and run:

```
cd ~/Downloads
gcc -Wall -Wextra -Wpedantic railway_reservation_v4_responsive_no_top_bar.c -o
railway_v4 \
    $(pkg-config --cflags --libs gtk+-3.0)
```

9.2 Running the application

The program is launched with a clean environment so that only the variables GTK needs (display, X authority, runtime directory, data and config directories) are passed through. This avoids conflicts with unrelated environment settings.

```
env -i HOME="$HOME" PATH="/usr/bin:/bin" DISPLAY="$DISPLAY" \
XAUTHORITY="$XAUTHORITY" XDG_RUNTIME_DIR="$XDG_RUNTIME_DIR" \
XDG_DATA_DIRS="/usr/local/share:/usr/share" \
XDG_CONFIG_DIRS="/etc/xdg" ./railway_v4
```

If the build fails because GTK is missing, install the toolchain with: sudo apt install build-essential pkg-config libgtk-3-dev.

10. Testing

The program was tested manually through the GUI. Each test below states the action, the expected behaviour that follows from the code, and what it demonstrates about the data structure. The program was also compiled with -Wall -Wextra -Wpedantic to catch questionable constructs at build time.

No.	Test action	Expected result	Concept verified
1	Book one Sleeper passenger with valid data	Confirmed, seat SL-01, Ticket 1001, e-ticket shown	enqueue into confirmed
2	Book 20 Sleeper passengers, then a 21st	21st goes to waiting list, WL 1	Per-class capacity; enqueue into waiting
3	Book 3 First-class passengers, then a 4th	4th waits although the train still has seats in other classes	Class limit vs. total limit
4	Cancel SL-05, with a waiting Sleeper passenger present	Waiting passenger promoted and given SL-05	remove_ticket + remove_first_class
5	Cancel a confirmed 3A ticket while only SL passengers wait	Seat released, nobody promoted	Same-class rule
6	Cancel a passenger in the middle of the waiting list	Those behind move up one position	Unlinking a middle node
7	Cancel the last waiting passenger, then book another	New passenger appended correctly	rear pointer fix-up
8	Cancel then book in the same class	Lowest free seat number reused	make_seat gap search

No.	Test action	Expected result	Concept verified
9	Fill 38 confirmed + 10 waiting, then book again	"Train and waiting list are full" error	Overflow handling
10	Empty name, short phone, e-mail without @, date 31-02-2026	Each rejected with a message; no record created	Validation
11	Search by Ticket ID, by PNR and by an unknown value	First two show details; last reports not found	find_ticket / find_pnr
12	Enter letters or 0 as Ticket ID for cancel	Rejected as invalid	parse_ticket_id
13	Use Reset System and confirm	All queues emptied, counters back to 1001	free_queue, re-initialisation
14	Toggle dark mode and confirmation switch	Theme changes immediately; dialogs suppressed when off	Settings

11. Output and Results

11.1 Summary of results

Feature	Result demonstrated by the program
FIFO booking	Passengers are confirmed in arrival order and receive the lowest free seat of their class.
Waiting list	When a class is full the passenger joins the waiting queue with a live position number.
Promotion	A cancelled confirmed seat passes to the first waiting passenger of the same class.
Seat reuse	Released seats are reused, so no seat is issued twice within a class.
Validation	Invalid name, phone, e-mail or date is rejected before any record is created.
Search	A passenger is found by Ticket ID or PNR and the details panel updates instantly.
Memory	Every removed node is freed; reset and exit release both queues completely.

11.2 Output screenshots

The screens below were captured from the executed program (Ubuntu, GTK3) immediately after launch, before any booking was made. They show the initial state of every page of the application: all 38 seats free, empty queues and the start-up log entries.

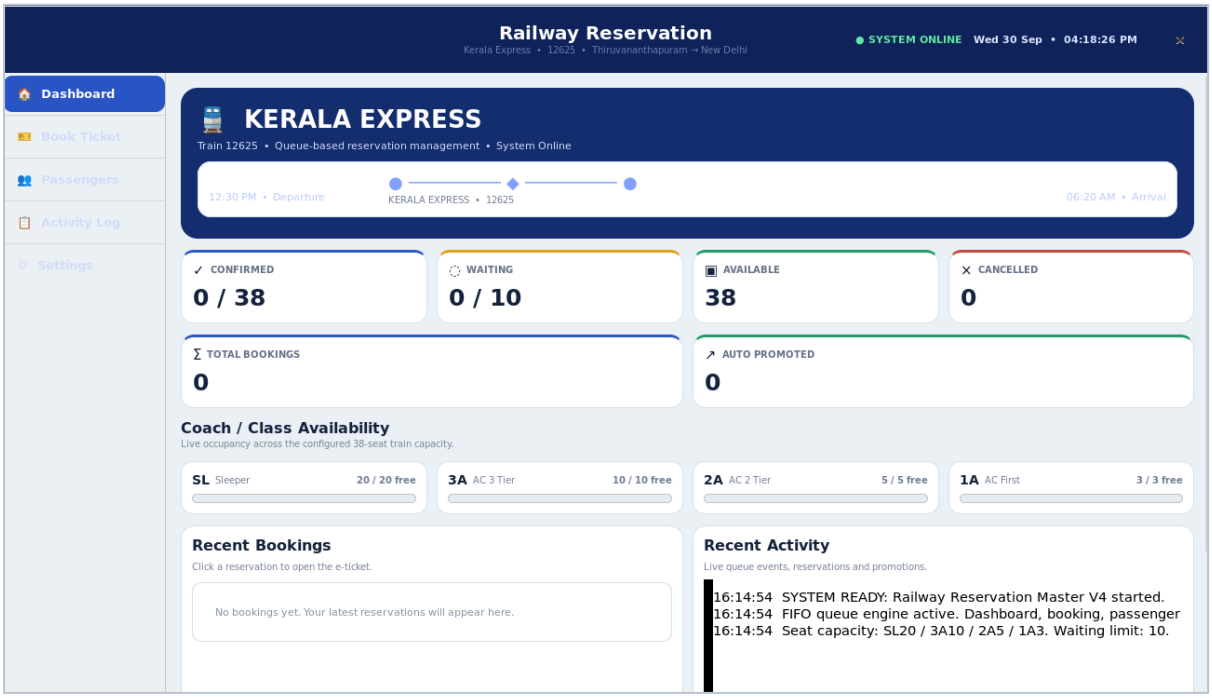


Figure 4: Dashboard with live statistics and class availability

Dashboard in its initial state: 0 / 38 confirmed, 0 / 10 waiting, 38 seats available and every class card at full availability (SL 20, 3A 10, 2A 5, 1A 3). The Recent Activity panel already shows the three start-up messages written by log_msg().

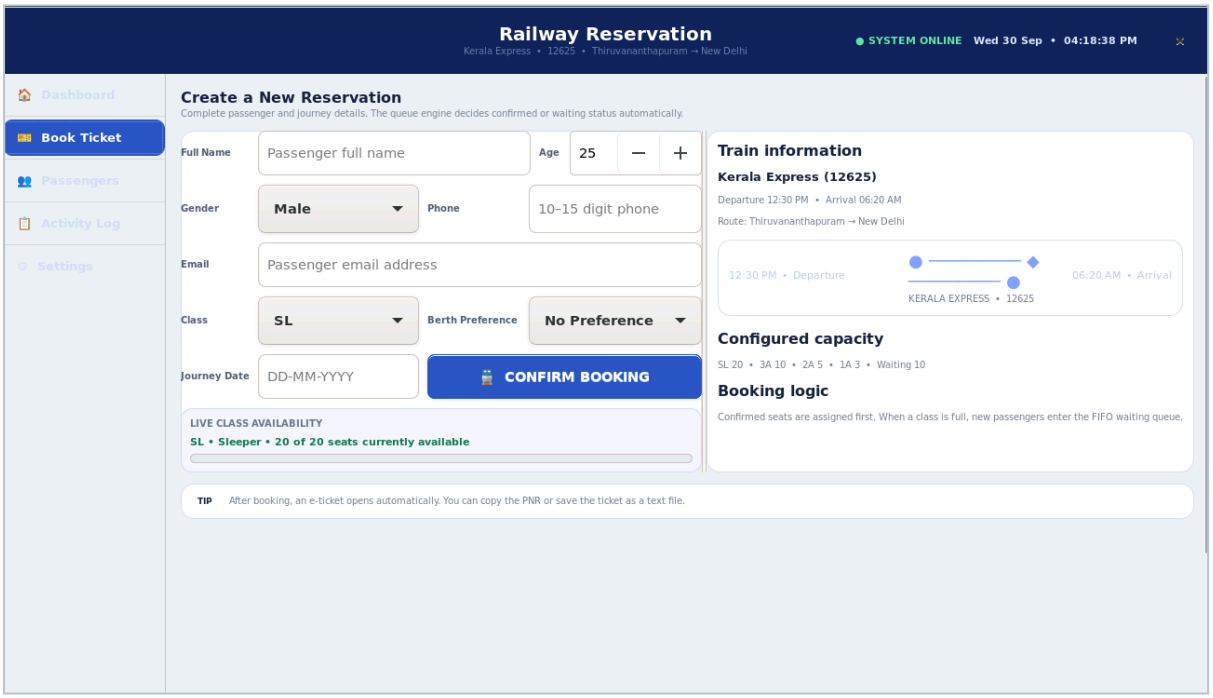


Figure 5: Booking form

Booking form with fields for name, age, gender, phone, e-mail, class, berth preference and journey date. The live availability bar reports 20 of 20 Sleeper seats free, and the right-hand panel shows the train details, configured capacity and booking logic.

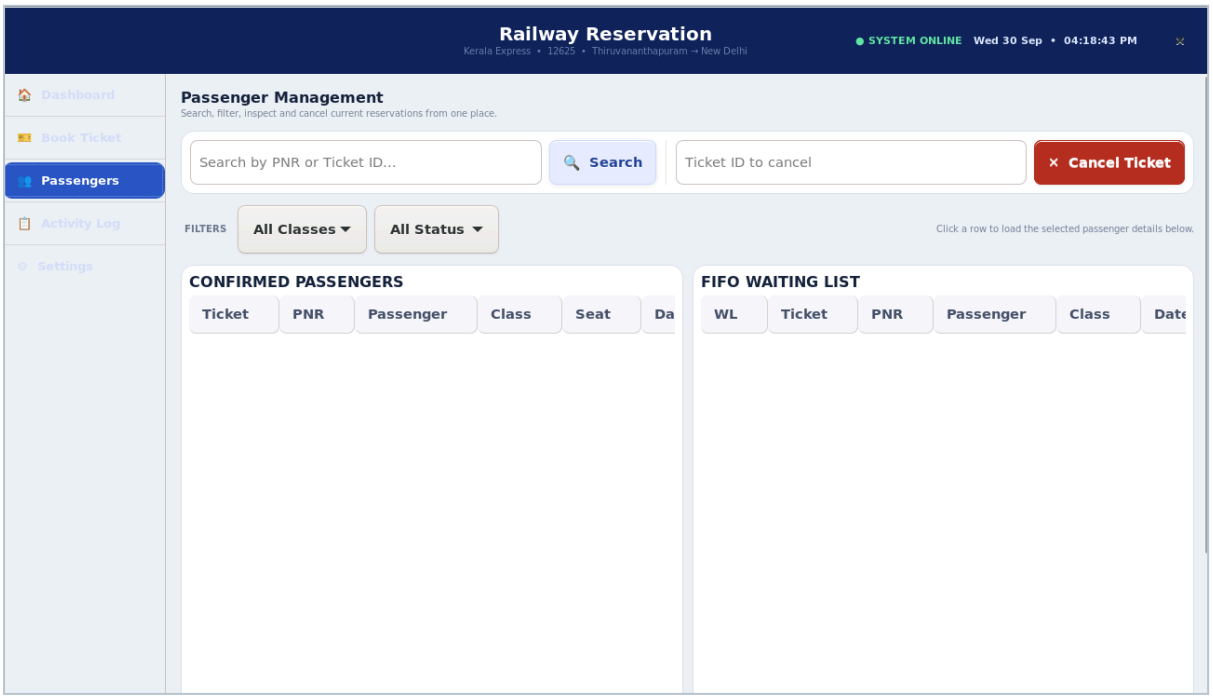


Figure 6: Passenger management with the confirmed and waiting queue tables

Passenger Management page with search by PNR or Ticket ID, cancellation by Ticket ID, class and status filters, and two separate tables for the confirmed queue and the FIFO waiting list. Both tables are empty because no reservation has been made yet.

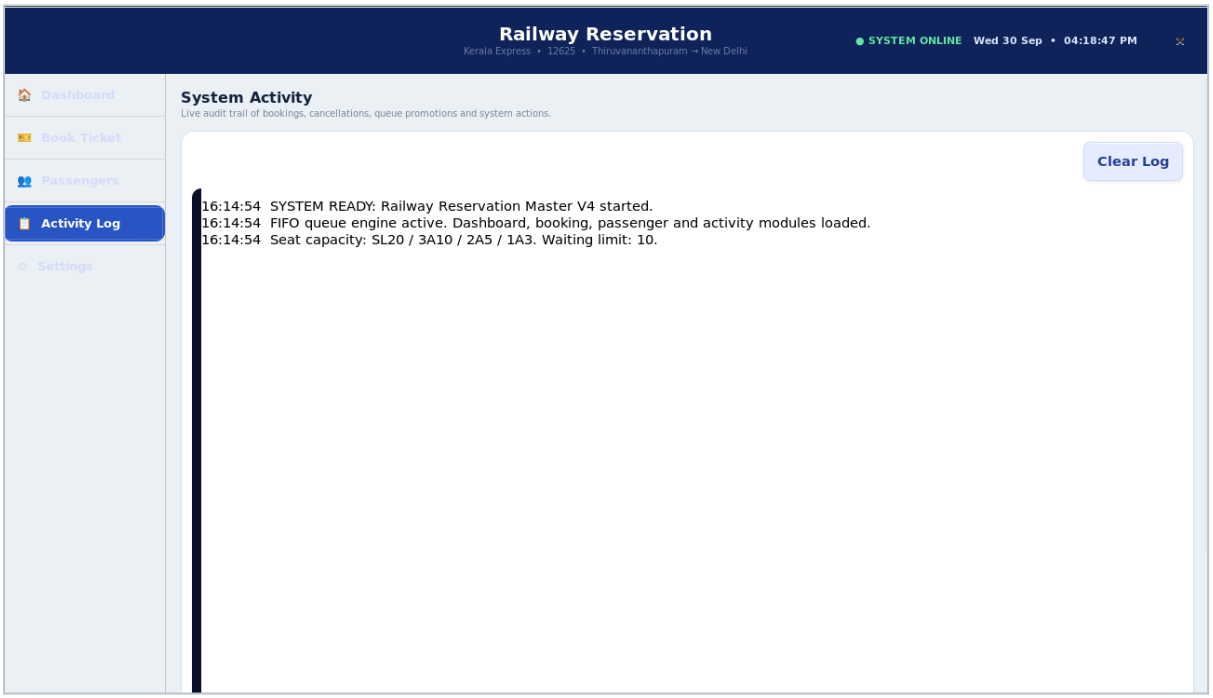


Figure 7: Activity log showing the system start-up events

Activity Log page. The timestamped entries record system start-up, activation of the FIFO queue engine and the configured seat capacities (SL 20, 3A 10, 2A 5, 1A 3, waiting limit 10). Bookings, cancellations and promotions are appended here as they occur.

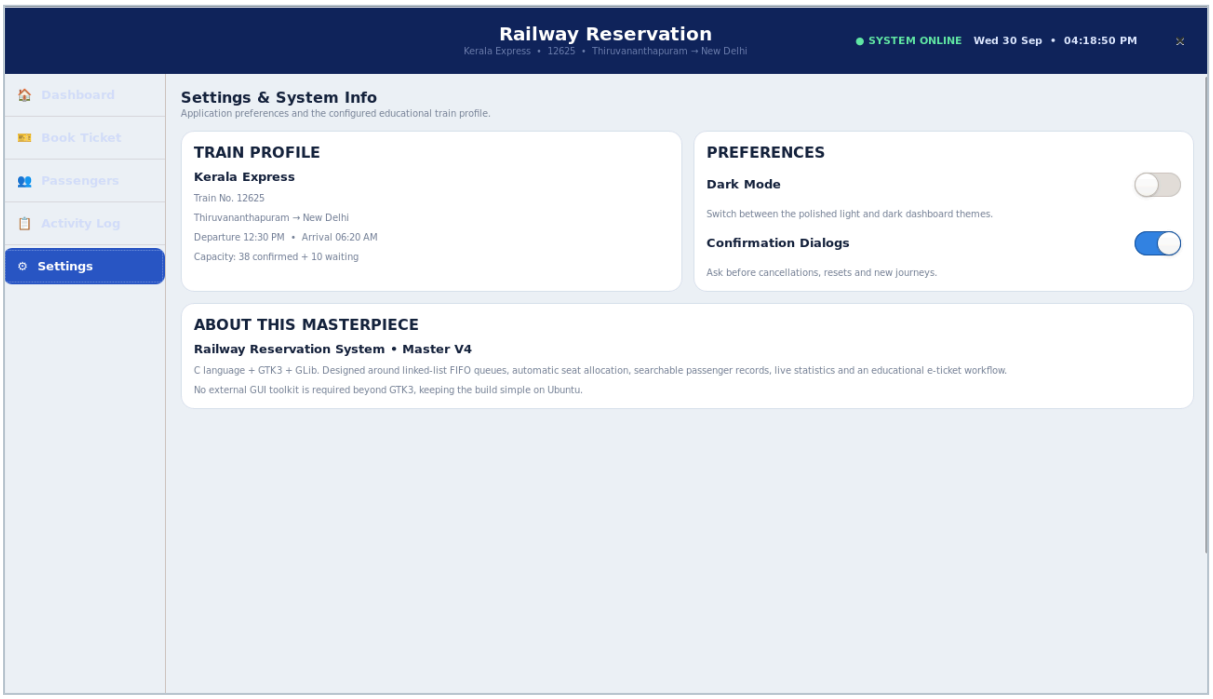


Figure 8: Settings and system information page

Settings page showing the train profile (Kerala Express, No. 12625, 38 confirmed + 10 waiting), the Dark Mode switch (off), the Confirmation Dialogs switch (on) and the project description.

12. Source Code: Complete Program Listing

File: railway_reservation_v4_responsive_no_top_bar.c (2688 lines). The listing below is the complete, unmodified source, divided into modules 12.1 to 12.12. Line numbers match the original file.

12.1 Header, macros, data structures and global state (lines 1 – 160)

```
/*
 * RAILWAY RESERVATION SYSTEM - MASTER V4
 * C + GTK3
 *
 * Build:
 *   gcc -Wall -Wextra -Wpedantic railway_reservation_master_v4.c -o railway_master \
 *       $(pkg-config --cflags --libs gtk+-3.0)
 *
 * A polished educational railway reservation dashboard built around
 * FIFO linked-list queues for confirmed and waiting passengers.
 *
 * Highlights:
 *   - Modern GTK3 HeaderBar + Stack + StackSidebar navigation
 *   - Responsive dashboard with live statistics
 *   - Coach/class availability cards and progress bars
 *   - Recent-booking cards and live activity timeline
 *   - Full booking form with email and berth preference
 *   - Search by Ticket ID or PNR
 *   - Class/status filters for passenger management
 *   - Selected-passenger details panel
 *   - Automatic seat allocation and same-class FIFO promotion
 *   - E-ticket dialog with Copy PNR and Save Ticket
 *   - Confirmation dialogs, status badges, notifications
 *   - Settings page with Dark Mode and confirmation preferences
 *   - No third-party GUI library beyond GTK3 / GLib
 */

#include <gtk/gtk.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdarg.h>
#include <errno.h>
#include <limits.h>
#include <time.h>
#include <ctype.h>

#define NAME_LEN       48
#define PHONE_LEN      20
#define EMAIL_LEN      80
#define FROM_LEN       32
#define TO_LEN         32
#define CLASS_LEN      8
#define GENDER_LEN     12
#define BERTH_LEN      24
#define PNR_LEN        20
#define DATE_LEN       20
#define SEAT_LEN       16

#define MAX_CONFIRMED  38
#define MAX_WAITING    10

#define SL_CAP         20
#define A3_CAP         10
#define A2_CAP         5
#define A1_CAP         3

typedef struct Passenger {
    int ticket_id;
    char pnr[PNR_LEN];
    char name[NAME_LEN];
    int age;
    char gender[GENDER_LEN];
    char phone[PHONE_LEN];
    char email[EMAIL_LEN];
    char travel_class[CLASS_LEN];
    char from[FROM_LEN];
```

```
    char to[TO_LEN];
    char date[DATE_LEN];
    char berth[BERTH_LEN];
    char seat[SEAT_LEN];
    int waiting_position;
    struct Passenger *next;
} Passenger;

typedef struct {
    Passenger *front;
    Passenger *rear;
    int size;
} Queue;

/* ---------- Global application state ---------- */

static Queue confirmed;
static Queue waiting;

static int next_ticket = 1001;
static unsigned long long next_pnr = 4817263951ULL;
static int total_bookings = 0;
static int total_cancelled = 0;
static int total_promoted = 0;

static gboolean dark_mode = FALSE;
static gboolean confirm_actions = TRUE;

static GtkWidget *window;
static GtkWidget *main_stack;
static GtkWidget *header_clock;
static GtkWidget *header_status;

/* Booking widgets */
static GtkWidget *entry_name;
static GtkWidget *spin_age;
static GtkWidget *combo_gender;
static GtkWidget *entry_phone;
static GtkWidget *entry_email;
static GtkWidget *combo_class;
static GtkWidget *combo_berth;
static GtkWidget *entry_date;
static GtkWidget *booking_availability_label;
static GtkWidget *booking_availability_progress;

/* Search/cancellation widgets */
static GtkWidget *entry_search;
static GtkWidget *entry_cancel;
static GtkWidget *combo_filter_class;
static GtkWidget *combo_filter_status;

/* Dashboard widgets */
static GtkWidget *lbl_confirmed;
static GtkWidget *lbl_waiting;
static GtkWidget *lbl_available;
static GtkWidget *lbl_cancelled;
static GtkWidget *dash_total;
static GtkWidget *dash_promoted;
static GtkWidget *class_free[4];
static GtkWidget *class_progress[4];
static GtkListBox *recent_list;
static GtkTextBuffer *dash_activity_buffer;

/* Passenger details */
static int selected_ticket_id = -1;
static GtkWidget *detail_status;
static GtkWidget *detail_name;
static GtkWidget *detail_ticket;
static GtkWidget *detail_pnr;
static GtkWidget *detail_class;
static GtkWidget *detail_seat;
static GtkWidget *detail_date;
static GtkWidget *detail_phone;
static GtkWidget *detail_email;
static GtkWidget *detail_berth;

/* Passenger tables */
static GtkListStore *store_confirmed;
static GtkListStore *store_waiting;

```

```
148 /* Activity */
149 static GtkWidget *log_view;
150 static GtkTextBuffer *log_buffer;
151 static GtkWidget *lbl_status;
152
153 /* Settings */
154 static GtkWidget *switch_dark;
155 static GtkWidget *switch_confirm;
156
157 /* Forward declarations for UI callbacks/helpers used before their definitions. */
158 static void show_notification(const char *message, GtkMessageType type);
159 static void show_ticket_dialog(Passenger *p);
160
```

12.2 Queue engine (lines 161 – 268)

```
161 /* ---------- Queue functions ---------- */
162
163 static void queue_init(Queue *q)
164 {
165     if (!q) return;
166     q->front = NULL;
167     q->rear = NULL;
168     q->size = 0;
169 }
170
171 static void enqueue(Queue *q, Passenger *p)
172 {
173     if (!q || !p) return;
174
175     p->next = NULL;
176     if (q->rear)
177         q->rear->next = p;
178     else
179         q->front = p;
180
181     q->rear = p;
182     q->size++;
183 }
184
185 static Passenger *dequeue(Queue *q)
186 {
187     Passenger *p;
188
189     if (!q || !q->front)
190         return NULL;
191
192     p = q->front;
193     q->front = p->next;
194     if (!q->front)
195         q->rear = NULL;
196
197     p->next = NULL;
198     q->size--;
199     return p;
200 }
201
202 static Passenger *remove_ticket(Queue *q, int ticket_id)
203 {
204     Passenger *prev = NULL;
205     Passenger *cur;
206
207     if (!q) return NULL;
208
209     cur = q->front;
210     while (cur) {
211         if (cur->ticket_id == ticket_id) {
212             if (prev)
213                 prev->next = cur->next;
214             else
215                 q->front = cur->next;
216
217             if (q->rear == cur)
218                 q->rear = prev;
219
220             cur->next = NULL;
221             q->size--;
222             return cur;
```

```
223         }
224         prev = cur;
225         cur = cur->next;
226     }
227
228     return NULL;
229 }
230
231 static Passenger *remove_first_class(Queue *q, const char *class_code)
232 {
233     Passenger *prev = NULL;
234     Passenger *cur;
235
236     if (!q || !class_code) return NULL;
237
238     cur = q->front;
239     while (cur) {
240         if (strcmp(cur->travel_class, class_code) == 0) {
241             if (prev)
242                 prev->next = cur->next;
243             else
244                 q->front = cur->next;
245
246             if (q->rear == cur)
247                 q->rear = prev;
248
249             cur->next = NULL;
250             q->size--;
251             return cur;
252         }
253         prev = cur;
254         cur = cur->next;
255     }
256
257     return NULL;
258 }
259
260 static void free_queue(Queue *q)
261 {
262     Passenger *p;
263
264     if (!q) return;
265     while ((p = dequeue(q)) != NULL)
266         free(p);
267 }
268
```

12.3 Utility and validation functions (lines 269 – 476)

```
269 /* ---------- Utility ---------- */
270
271 static void copy_text(char *dest, size_t dest_size, const char *src)
272 {
273     if (!dest || dest_size == 0) return;
274     if (!src) {
275         dest[0] = '\0';
276         return;
277     }
278     snprintf(dest, dest_size, "%s", src);
279 }
280
281 static void trim_spaces(char *s)
282 {
283     char *start;
284     char *end;
285
286     if (!s) return;
287
288     start = s;
289     while (*start == ' ' || *start == '\t' || *start == '\r' || *start == '\n')
290         start++;
291
292     if (start != s)
293         memmove(s, start, strlen(start) + 1);
294
295     if (*s == '\0') return;
296
297     end = s + strlen(s) - 1;
```

```
    while (end >= s && (*end == ' ' || *end == '\t' || *end == '\r' || *end == '\n')) {
        *end = '\0';
        end--;
    }
}

static int class_capacity(const char *code)
{
    if (strcmp(code, "SL") == 0) return SL_CAP;
    if (strcmp(code, "3A") == 0) return A3_CAP;
    if (strcmp(code, "2A") == 0) return A2_CAP;
    if (strcmp(code, "1A") == 0) return A1_CAP;
    return 0;
}

static const char *class_name(const char *code)
{
    if (strcmp(code, "SL") == 0) return "Sleeper";
    if (strcmp(code, "3A") == 0) return "AC 3 Tier";
    if (strcmp(code, "2A") == 0) return "AC 2 Tier";
    if (strcmp(code, "1A") == 0) return "AC First";
    return "Unknown";
}

static int confirmed_class_count(const char *code)
{
    int count = 0;
    Passenger *p;

    for (p = confirmed.front; p; p = p->next) {
        if (strcmp(p->travel_class, code) == 0)
            count++;
    }
    return count;
}

static int available_total(void)
{
    return MAX_CONFIRMED - confirmed.size;
}

static Passenger *find_ticket(int ticket_id)
{
    Passenger *p;

    for (p = confirmed.front; p; p = p->next)
        if (p->ticket_id == ticket_id) return p;

    for (p = waiting.front; p; p = p->next)
        if (p->ticket_id == ticket_id) return p;

    return NULL;
}

static Passenger *find_pnr(const char *pnr)
{
    Passenger *p;

    if (!pnr) return NULL;

    for (p = confirmed.front; p; p = p->next)
        if (strcmp(p->pnr, pnr) == 0) return p;

    for (p = waiting.front; p; p = p->next)
        if (strcmp(p->pnr, pnr) == 0) return p;

    return NULL;
}

static void make_pnr(char *out, size_t size)
{
    snprintf(out, size, "%010llu", next_pnr++);
}

static void make_seat(char *out, size_t size, const char *code)
{
    int capacity = class_capacity(code);
    int used[SL_CAP + 1] = {0};
    int seat_number;
    Passenger *p;
```

```
    for (p = confirmed.front; p; p = p->next) {
        if (strcmp(p->travel_class, code) == 0 &&
            strncmp(p->seat, code, strlen(code)) == 0) {
            int n = 0;
            if (sscanf(p->seat + strlen(code), "-%d", &n) == 1 &&
                n >= 1 && n <= capacity)
                used[n] = 1;
        }
    }

    for (seat_number = 1; seat_number <= capacity; seat_number++) {
        if (!used[seat_number]) {
            snprintf(out, size, "%s-%02d", code, seat_number);
            return;
        }
    }

    out[0] = '\0';
}

static int parse_ticket_id(const char *text, int *out)
{
    char *end;
    long value;

    if (!text || !out) return 0;

    errno = 0;
    value = strtol(text, &end, 10);

    while (*end == ' ' || *end == '\t')
        end++;

    if (text[0] == '\0' || *end != '\0' || errno == ERANGE ||
        value < 1 || value > INT_MAX)
        return 0;

    *out = (int)value;
    return 1;
}

static int valid_date(const char *text)
{
    int day, month, year;
    int max_day;

    if (!text || strlen(text) != 10)
        return 0;

    if (text[2] != '-' || text[5] != '-')
        return 0;

    if (sscanf(text, "%2d-%2d-%4d", &day, &month, &year) != 3)
        return 0;

    if (year < 2024 || month < 1 || month > 12 || day < 1)
        return 0;

    max_day = 31;
    if (month == 4 || month == 6 || month == 9 || month == 11)
        max_day = 30;
    else if (month == 2) {
        int leap = (year % 400 == 0) || (year % 4 == 0 && year % 100 != 0);
        max_day = leap ? 29 : 28;
    }

    return day <= max_day;
}

static int valid_phone(const char *text)
{
    size_t i;
    size_t digits = 0;

    if (!text) return 0;

    for (i = 0; text[i] != '\0'; i++) {
        if (isdigit((unsigned char)text[i]))
            digits++;
```

```
458         else if (text[i] != '+' && text[i] != '-' && text[i] != ' ' && text[i] != '(' && text[i] != ')')
459             return 0;
460     }
461
462     return digits >= 10 && digits <= 15;
463 }
464
465 static int valid_email(const char *text)
466 {
467     const char *at;
468     const char *dot;
469
470     if (!text || text[0] == '\0') return 0;
471     at = strchr(text, '@');
472     dot = strrchr(text, '.');
473
474     return at && dot && at < dot && at != text && dot[1] != '\0';
475 }
476
```

12.4 Logging (lines 477 – 515)

```
477 /* ---------- Logging ---------- */
478
479 static void append_activity(GtkTextBuffer *buffer, const char *line)
480 {
481     GtkTextIter end;
482     if (!buffer) return;
483     gtk_text_buffer_get_end_iter(buffer, &end);
484     gtk_text_buffer_insert(buffer, &end, line, -1);
485 }
486
487 static void log_msg(const char *fmt, ...)
488 {
489     char msg[512];
490     char line[640];
491     char time_text[32];
492     time_t now;
493     struct tm *tm_info;
494     va_list ap;
495
496     va_start(ap, fmt);
497     vsnprintf(msg, sizeof msg, fmt, ap);
498     va_end(ap);
499
500     now = time(NULL);
501     tm_info = localtime(&now);
502     if (tm_info)
503         strftime(time_text, sizeof time_text, "%H:%M:%S", tm_info);
504     else
505         snprintf(time_text, sizeof time_text, "--:--:--");
506
507     snprintf(line, sizeof line, "%s  %s\n", time_text, msg);
508
509     append_activity(log_buffer, line);
510     append_activity(dash_activity_buffer, line);
511
512     if (lbl_status)
513         gtk_label_set_text(GTK_LABEL(lbl_status), msg);
514 }
515
```

12.5 UI helper functions (lines 516 – 625)

```
516 /* ---------- UI helpers ---------- */
517
518 static GtkWidget *make_label(const char *text, const char *class_name_text)
519 {
520     GtkWidget *label = gtk_label_new(text);
521     gtk_label_set_xalign(GTK_LABEL(label), 0.0f);
522
523     if (class_name_text)
524         gtk_style_context_add_class(gtk_widget_get_style_context(label), class_name_text);
525
526     return label;
527 }
```

```
528
529 static GtkWidget *make_card(void)
530 {
531     GtkWidget *box = gtk_event_box_new();
532     gtk_style_context_add_class(gtk_widget_get_style_context(box), "card");
533     return box;
534 }
535
536 static GtkWidget *make_stat_card(const char *icon,
537                                  const char *title,
538                                  GtkWidget **value,
539                                  const char *accent_class)
540 {
541     GtkWidget *event = make_card();
542     GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 5);
543     GtkWidget *top = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 6);
544     GtkWidget *icon_label = make_label(icon, "stat-icon");
545     GtkWidget *title_label = make_label(title, "stat-title");
546     GtkWidget *number = make_label("0", "stat-value");
547
548     gtk_style_context_add_class(gtk_widget_get_style_context(event), accent_class);
549     gtk_container_add(GTK_CONTAINER(event), box);
550     gtk_container_set_border_width(GTK_CONTAINER(box), 13);
551
552     gtk_box_pack_start(GTK_BOX(top), icon_label, FALSE, FALSE, 0);
553     gtk_box_pack_start(GTK_BOX(top), title_label, TRUE, TRUE, 0);
554     gtk_box_pack_start(GTK_BOX(box), top, FALSE, FALSE, 0);
555     gtk_box_pack_start(GTK_BOX(box), number, FALSE, FALSE, 0);
556
557     *value = number;
558     return event;
559 }
560
561 static GtkWidget *make_class_card(const char *code, const char *name, int index)
562 {
563     GtkWidget *event = make_card();
564     GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 6);
565     GtkWidget *head = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 6);
566     GtkWidget *code_label = make_label(code, "class-code");
567     GtkWidget *name_label = make_label(name, "mini");
568     GtkWidget *free_label = make_label("0 / 0 free", "class-free");
569     GtkWidget *progress = gtk_progress_bar_new();
570
571     gtk_style_context_add_class(gtk_widget_get_style_context(event), "class-card");
572     gtk_container_add(GTK_CONTAINER(event), box);
573     gtk_container_set_border_width(GTK_CONTAINER(box), 12);
574
575     gtk_box_pack_start(GTK_BOX(head), code_label, FALSE, FALSE, 0);
576     gtk_box_pack_start(GTK_BOX(head), name_label, TRUE, TRUE, 0);
577     gtk_box_pack_end(GTK_BOX(head), free_label, FALSE, FALSE, 0);
578     gtk_box_pack_start(GTK_BOX(box), head, FALSE, FALSE, 0);
579
580     gtk_progress_bar_set_show_text(GTK_PROGRESS_BAR(progress), FALSE);
581     gtk_box_pack_start(GTK_BOX(box), progress, FALSE, FALSE, 0);
582
583     class_free[index] = free_label;
584     class_progress[index] = progress;
585     return event;
586 }
587
588 static GtkWidget *make_section_header(const char *title, const char *subtitle)
589 {
590     GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 1);
591     gtk_box_pack_start(GTK_BOX(box), make_label(title, "section-title"), FALSE, FALSE, 0);
592     if (subtitle)
593         gtk_box_pack_start(GTK_BOX(box), make_label(subtitle, "section-subtitle"), FALSE, FALSE, 0);
594     return box;
595 }
596
597 static GtkWidget *make_page(void)
598 {
599     GtkWidget *page = gtk_box_new(GTK_ORIENTATION_VERTICAL, 12);
600     gtk_style_context_add_class(gtk_widget_get_style_context(page), "page");
601     return page;
602 }
603
604 static GtkWidget *make_field_label(const char *text)
605 {
606     GtkWidget *label = gtk_label_new(text);
607     gtk_label_set_xalign(GTK_LABEL(label), 0.0f);
```

```
608     gtk_style_context_add_class(gtk_widget_get_style_context(label), "field-label");
609     return label;
610 }
611 
612 static GtkWidget *make_badge(const char *text, const char *class_name_text)
613 {
614     GtkWidget *label = gtk_label_new(text);
615     gtk_style_context_add_class(gtk_widget_get_style_context(label), "badge");
616     gtk_style_context_add_class(gtk_widget_get_style_context(label), class_name_text);
617     return label;
618 }
619 
620 static void set_detail_text(GtkWidget *label, const char *text)
621 {
622     if (label)
623         gtk_label_set_text(GTK_LABEL(label), text ? text : "—");
624 }
625 
```

12.6 Dashboard, filters and table refresh (lines 626 – 993)

```
626 /* ---------- Dashboard ---------- */
627 
628 static void refresh_dashboard(void)
629 {
630     const char *codes[] = {"SL", "3A", "2A", "1A"};
631     int i;
632     char text[128];
633 
634     snprintf(text, sizeof text, "%d / %d", confirmed.size, MAX_CONFIRMED);
635     gtk_label_set_text(GTK_LABEL(lbl_confirmed), text);
636 
637     snprintf(text, sizeof text, "%d / %d", waiting.size, MAX_WAITING);
638     gtk_label_set_text(GTK_LABEL(lbl_waiting), text);
639 
640     snprintf(text, sizeof text, "%d", available_total());
641     gtk_label_set_text(GTK_LABEL(lbl_available), text);
642 
643     snprintf(text, sizeof text, "%d", total_cancelled);
644     gtk_label_set_text(GTK_LABEL(lbl_cancelled), text);
645 
646     snprintf(text, sizeof text, "%d", total_bookings);
647     gtk_label_set_text(GTK_LABEL(dash_total), text);
648 
649     snprintf(text, sizeof text, "%d", total_promoted);
650     gtk_label_set_text(GTK_LABEL(dash_promoted), text);
651 
652     for (i = 0; i < 4; i++) {
653         int cap = class_capacity(codes[i]);
654         int used = confirmed_class_count(codes[i]);
655         int free_seats = cap - used;
656         double fraction = cap > 0 ? (double)used / (double)cap : 0.0;
657         char free_text[64];
658 
659         snprintf(free_text, sizeof free_text, "%d / %d free", free_seats, cap);
660         gtk_label_set_text(GTK_LABEL(class_free[i]), free_text);
661         gtk_progress_bar_set_fraction(GTK_PROGRESS_BAR(class_progress[i]), fraction);
662     }
663 }
664 
665 static int passes_class_filter(const char *code)
666 {
667     gchar *selected;
668     int pass;
669 
670     if (!combo_filter_class) return 1;
671     selected = gtk_combo_box_text_get_active_text(GTK_COMBO_BOX_TEXT(combo_filter_class));
672     if (!selected) return 1;
673 
674     pass = (strcmp(selected, "All Classes") == 0 || strcmp(selected, code) == 0);
675     g_free(selected);
676     return pass;
677 }
678 
679 static int passes_status_filter(int is_waiting)
680 {
681     gchar *selected;
682     int pass;
```

```
683
684     if (!combo_filter_status) return 1;
685     selected = gtk_combo_box_text_get_active_text(GTK_COMBO_BOX_TEXT(combo_filter_status));
686     if (!selected) return 1;
687
688     if (strcmp(selected, "All Status") == 0)
689         pass = 1;
690     else if (strcmp(selected, "Confirmed") == 0)
691         pass = !is_waiting;
692     else if (strcmp(selected, "Waiting") == 0)
693         pass = is_waiting;
694     else
695         pass = 1;
696
697     g_free(selected);
698     return pass;
699 }
700
701 static void set_detail_status_text(const char *text, gboolean is_waiting)
702 {
703     GtkStyleContext *ctx;
704
705     if (!detail_status) return;
706
707     gtk_label_set_text(GTK_LABEL(detail_status), text ? text : "—");
708     ctx = gtk_widget_get_style_context(detail_status);
709     gtk_style_context_remove_class(ctx, "status-confirmed");
710     gtk_style_context_remove_class(ctx, "status-waiting");
711     if (text && is_waiting)
712         gtk_style_context_add_class(ctx, "status-waiting");
713     else if (text)
714         gtk_style_context_add_class(ctx, "status-confirmed");
715 }
716
717 static void refresh_selected_details_real(void)
718 {
719     Passenger *p = NULL;
720     char text[128];
721     gboolean is_waiting;
722
723     if (selected_ticket_id > 0)
724         p = find_ticket(selected_ticket_id);
725
726     if (!p) {
727         set_detail_text(detail_name, "Select a passenger from the table");
728         set_detail_text(detail_ticket, "—");
729         set_detail_text(detail_pnr, "—");
730         set_detail_text(detail_class, "—");
731         set_detail_text(detail_seat, "—");
732         set_detail_text(detail_date, "—");
733         set_detail_text(detail_phone, "—");
734         set_detail_text(detail_email, "—");
735         set_detail_text(detail_berth, "—");
736         set_detail_status_text("—", FALSE);
737         return;
738     }
739
740     is_waiting = (p->seat[0] == '\0');
741     set_detail_text(detail_name, p->name);
742     snprintf(text, sizeof text, "%d", p->ticket_id);
743     set_detail_text(detail_ticket, text);
744     set_detail_text(detail_pnr, p->pnr);
745     set_detail_text(detail_class, p->travel_class);
746     set_detail_text(detail_seat, is_waiting ? "Waiting List" : p->seat);
747     set_detail_text(detail_date, p->date);
748     set_detail_text(detail_phone, p->phone);
749     set_detail_text(detail_email, p->email);
750     set_detail_text(detail_berth, p->berth);
751     set_detail_status_text(is_waiting ? "WAITING" : "CONFIRMED", is_waiting);
752 }
753
754 /* Keep the public name small so refresh callers stay readable. */
755 static void refresh_details(void)
756 {
757     refresh_selected_details_real();
758 }
759
760 static void clear_recent_list(void)
761 {
762     GList *children;
```

```
    GList *iter;

    if (!recent_list) return;

    children = gtk_container_get_children(GTK_CONTAINER(recent_list));
    for (iter = children; iter; iter = iter->next)
        gtk_widget_destroy(GTK_WIDGET(iter->data));
    g_list_free(children);
}

static int cmp_recent_desc(const void *a, const void *b)
{
    const Passenger *pa = *(const Passenger * const *)a;
    const Passenger *pb = *(const Passenger * const *)b;

    if (pa->ticket_id < pb->ticket_id) return 1;
    if (pa->ticket_id > pb->ticket_id) return -1;
    return 0;
}

static GtkWidget *make_recent_row(Passenger *p)
{
    GtkWidget *row = gtk_list_box_row_new();
    GtkWidget *outer = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    GtkWidget *ticket_box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 0);
    GtkWidget *ticket = gtk_label_new(NULL);
    GtkWidget *name = gtk_label_new(p->name);
    GtkWidget *middle = gtk_box_new(GTK_ORIENTATION_VERTICAL, 1);
    GtkWidget *sub = gtk_label_new(NULL);
    GtkWidget *right = gtk_box_new(GTK_ORIENTATION_VERTICAL, 1);
    GtkWidget *status;
    GtkWidget *seat;
    char buf[128];
    gboolean is_waiting = (p->seat[0] == '\0');

    gtk_style_context_add_class(gtk_widget_get_style_context(row), "recent-row");
    gtk_container_add(GTK_CONTAINER(row), outer);
    gtk_container_set_border_width(GTK_CONTAINER(outer), 10);

    snprintf(buf, sizeof buf, "#%d", p->ticket_id);
    gtk_label_set_text(GTK_LABEL(ticket), buf);
    gtk_label_set_xalign(GTK_LABEL(ticket), 0.0f);
    gtk_style_context_add_class(gtk_widget_get_style_context(ticket), "recent-ticket");
    gtk_box_pack_start(GTK_BOX(ticket_box), ticket, FALSE, FALSE, 0);

    gtk_label_set_xalign(GTK_LABEL(name), 0.0f);
    gtk_style_context_add_class(gtk_widget_get_style_context(name), "recent-name");
    gtk_box_pack_start(GTK_BOX(middle), name, FALSE, FALSE, 0);

    snprintf(buf, sizeof buf, "PNR %s  •  %s  •  %s", p->pnr,
             p->travel_class, p->date);
    gtk_label_set_text(GTK_LABEL(sub), buf);
    gtk_label_set_xalign(GTK_LABEL(sub), 0.0f);
    gtk_style_context_add_class(gtk_widget_get_style_context(sub), "mini");
    gtk_box_pack_start(GTK_BOX(middle), sub, FALSE, FALSE, 0);
    gtk_widget_set_hexpand(middle, TRUE);

    status = make_badge(is_waiting ? "WAITING" : "CONFIRMED",
                        is_waiting ? "badge-waiting" : "badge-confirmed");
    gtk_box_pack_start(GTK_BOX(right), status, FALSE, FALSE, 0);

    seat = gtk_label_new(NULL);
    snprintf(buf, sizeof buf, "%s", is_waiting ? "WL queue" : p->seat);
    gtk_label_set_text(GTK_LABEL(seat), buf);
    gtk_label_set_xalign(GTK_LABEL(seat), 1.0f);
    gtk_style_context_add_class(gtk_widget_get_style_context(seat), "mini");
    gtk_box_pack_start(GTK_BOX(right), seat, FALSE, FALSE, 0);

    gtk_box_pack_start(GTK_BOX(outer), ticket_box, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(outer), middle, TRUE, TRUE, 0);
    gtk_box_pack_end(GTK_BOX(outer), right, FALSE, FALSE, 0);

    g_object_set_data(G_OBJECT(row), "ticket-id", GINT_TO_POINTER(p->ticket_id));
    return row;
}

static void on_recent_activated(GtkListBox *list, GtkListBoxRow *row, gpointer data)
{
    int ticket_id;
    Passenger *p;
```

```
843
844     (void)list;
845     (void)data;
846     if (!row) return;
847
848     ticket_id = GPOINTER_TO_INT(g_object_get_data(G_OBJECT(row), "ticket-id"));
849     p = find_ticket(ticket_id);
850     if (p)
851         show_ticket_dialog(p);
852 }
853
854 static void refresh_recent_list(void)
855 {
856     Passenger *items[MAX_CONFIRMED + MAX_WAITING];
857     Passenger *p;
858     size_t count = 0;
859     size_t i;
860     GtkWidget *row;
861
862     clear_recent_list();
863
864     for (p = confirmed.front; p && count < G_N_ELEMENTS(items); p = p->next)
865         items[count++] = p;
866     for (p = waiting.front; p && count < G_N_ELEMENTS(items); p = p->next)
867         items[count++] = p;
868
869     if (count > 1)
870         qsort(items, count, sizeof items[0], cmp_recent_desc);
871
872     if (count == 0) {
873         GtkWidget *empty = gtk_label_new("No bookings yet. Your latest reservations will appear here.");
874         gtk_label_set_xalign(GTK_LABEL(empty), 0.0f);
875         gtk_style_context_add_class(gtk_widget_get_style_context(empty), "empty-text");
876         gtk_list_box_insert(recent_list, empty, -1);
877     } else {
878         size_t limit = count < 6 ? count : 6;
879         for (i = 0; i < limit; i++) {
880             row = make_recent_row(items[i]);
881             gtk_list_box_insert(recent_list, row, -1);
882         }
883     }
884
885     gtk_widget_show_all(GTK_WIDGET(recent_list));
886 }
887
888 static void refresh_tables(void)
889 {
890     GtkTreeIter iter;
891     Passenger *p;
892     int position = 1;
893
894     gtk_list_store_clear(store_confirmed);
895     gtk_list_store_clear(store_waiting);
896
897     if (passes_status_filter(0)) {
898         for (p = confirmed.front; p; p = p->next) {
899             if (!passes_class_filter(p->travel_class))
900                 continue;
901
902             gtk_list_store_append(store_confirmed, &iter);
903             gtk_list_store_set(store_confirmed, &iter,
904                                0, p->ticket_id,
905                                1, p->pnr,
906                                2, p->name,
907                                3, p->travel_class,
908                                4, p->seat,
909                                5, p->date,
910                                6, "CONFIRMED",
911                                -1);
912         }
913     }
914
915     for (p = waiting.front; p; p = p->next, position++)
916         p->waiting_position = position;
917
918     if (passes_status_filter(1)) {
919         position = 1;
920         for (p = waiting.front; p; p = p->next, position++) {
921             if (!passes_class_filter(p->travel_class))
922                 continue;
```

```
923
924             gtk_list_store_append(store_waiting, &iter);
925             gtk_list_store_set(store_waiting, &iter,
926                                0, position,
927                                1, p->ticket_id,
928                                2, p->pnr,
929                                3, p->name,
930                                4, p->travel_class,
931                                5, p->date,
932                                6, "WAITING",
933                                -1);
934         }
935     }
936
937     refresh_dashboard();
938     refresh_recent_list();
939     refresh_details();
940 }
941
942 static void update_booking_availability(void)
943 {
944     gchar *class_code;
945     int cap;
946     int used;
947     int free_seats;
948     char text[160];
949     double fraction;
950
951     if (!combo_class || !booking_availability_label || !booking_availability_progress)
952         return;
953
954     class_code = gtk_combo_box_text_get_active_text(GTK_COMBO_BOX_TEXT(combo_class));
955     if (!class_code) return;
956
957     cap = class_capacity(class_code);
958     used = confirmed_class_count(class_code);
959     free_seats = cap - used;
960     fraction = cap > 0 ? (double)used / (double)cap : 0.0;
961
962     if (free_seats > 0) {
963         snprintf(text, sizeof text,
964                  "%s • %s • %d of %d seats currently available",
965                  class_code, class_name(class_code), free_seats, cap);
966         gtk_style_context_remove_class(
967             gtk_widget_get_style_context(booking_availability_label), "warning-text");
968         gtk_style_context_add_class(
969             gtk_widget_get_style_context(booking_availability_label), "success-text");
970     } else {
971         snprintf(text, sizeof text,
972                  "%s • %s • CLASS FULL — a new booking will enter the FIFO waiting queue",
973                  class_code, class_name(class_code));
974         gtk_style_context_remove_class(
975             gtk_widget_get_style_context(booking_availability_label), "success-text");
976         gtk_style_context_add_class(
977             gtk_widget_get_style_context(booking_availability_label), "warning-text");
978     }
979
980     gtk_label_set_text(GTK_LABEL(booking_availability_label), text);
981     gtk_progress_bar_set_fraction(GTK_PROGRESS_BAR(booking_availability_progress), fraction);
982     g_free(class_code);
983 }
984
985 static void clear_booking_fields(void)
986 {
987     gtk_entry_set_text(GTK_ENTRY(entry_name), "");
988     gtk_entry_set_text(GTK_ENTRY(entry_phone), "");
989     gtk_entry_set_text(GTK_ENTRY(entry_email), "");
990     gtk_entry_set_text(GTK_ENTRY(entry_date), "");
991     gtk_widget_grab_focus(entry_name);
992 }
993
```

12.7 E-ticket generation, saving and dialog (lines 994 – 1200)

```
994 static gchar *ticket_text(Passenger *p)
995 {
996     const char *status = p->seat[0] ? "CONFIRMED" : "WAITING";
997     const char *seat_or_wl = p->seat[0] ? p->seat : "Waiting List";
```

```
    char wl_line[64] = "";

    if (!p->seat[0])
        snprintf(wl_line, sizeof wl_line, "Waiting Position : WL %d\n", p->waiting_position);

    return g_strdup_printf(
        "================================================\n"
        "           RAILWAY E-TICKET\n"
        "================================================\n"
        "PNR              : %s\n"
        "Ticket ID        : %d\n"
        "Status            : %s\n"
        "%s"
        "------------------------------------------------\n"
        "Train              : Kerala Express\n"
        "Train No.          : 12625\n"
        "From               : %s\n"
        "To                 : %s\n"
        "Journey Date       : %s\n"
        "Departure          : 12:30 PM\n"
        "Arrival            : 06:20 AM\n"
        "------------------------------------------------\n"
        "Passenger          : %s\n"
        "Age / Gender       : %d / %s\n"
        "Phone              : %s\n"
        "Email              : %s\n"
        "Class              : %s (%s)\n"
        "Berth Preference   : %s\n"
        "Seat               : %s\n"
        "================================================\n"
        "Educational simulation: FIFO queue reservation.\n"
        "Berth preference does not guarantee a real railway\n"
        "coach/berth allocation in this academic project.\n"
        "================================================\n",
        p->pnr, p->ticket_id, status, wl_line,
        p->from, p->to, p->date,
        p->name, p->age, p->gender, p->phone, p->email,
        p->travel_class, class_name(p->travel_class), p->berth, seat_or_wl);
}

static void save_ticket_to_file(Passenger *p, GtkWindow *parent)
{
    GtkWidget *dialog;
    GtkFileFilter *filter;
    gchar *text;
    gchar *path = NULL;
    GError *error = NULL;
    int response;
    char filename[64];

    if (!p) return;

    dialog = gtk_file_chooser_dialog_new(
        "Save Railway Ticket",
        parent,
        GTK_FILE_CHOOSER_ACTION_SAVE,
        "Cancel", GTK_RESPONSE_CANCEL,
        "Save", GTK_RESPONSE_ACCEPT,
        NULL);

    snprintf(filename, sizeof filename, "Railway_Ticket_%d.txt", p->ticket_id);
    gtk_file_chooser_set_current_name(GTK_FILE_CHOOSER(dialog), filename);

    filter = gtk_file_filter_new();
    gtk_file_filter_set_name(filter, "Text files");
    gtk_file_filter_add_pattern(filter, "*.txt");
    gtk_file_chooser_add_filter(GTK_FILE_CHOOSER(dialog), filter);

    response = gtk_dialog_run(GTK_DIALOG(dialog));
    if (response == GTK_RESPONSE_ACCEPT)
        path = gtk_file_chooser_get_filename(GTK_FILE_CHOOSER(dialog));

    gtk_widget_destroy(dialog);

    if (!path)
        return;

    text = ticket_text(p);
    if (!g_file_set_contents(path, text, -1, &error)) {
        log_msg("ERROR: could not save ticket: %s", error ? error->message : "unknown error");
```

```
        if (error) g_error_free(error);
    } else {
        log_msg("TICKET SAVED: #%d → %s", p->ticket_id, path);
        show_notification("Ticket saved successfully.", GTK_MESSAGE_INFO);
    }

    g_free(text);
    g_free(path);
}

static void show_ticket_dialog(Passenger *p)
{
    GtkWidget *dialog;
    GtkWidget *content;
    GtkWidget *top;
    GtkWidget *grid;
    GtkWidget *title;
    GtkWidget *status;
    GtkWidget *note;
    GtkWidget *copy_button;
    GtkWidget *save_button;
    GtkWidget *close_button;
    char text[256];
    int response;
    gboolean is_waiting;

    if (!p) return;

    is_waiting = (p->seat[0] == '\0');
    dialog = gtk_dialog_new();
    gtk_window_set_transient_for(GTK_WINDOW(dialog), GTK_WINDOW(window));
    gtk_window_set_modal(GTK_WINDOW(dialog), TRUE);
    gtk_window_set_title(GTK_WINDOW(dialog), is_waiting ? "Waiting List Ticket" : "Railway E-Ticket");
    gtk_window_set_default_size(GTK_WINDOW(dialog), 560, 620);

    content = gtk_dialog_get_content_area(GTK_DIALOG(dialog));
    gtk_container_set_border_width(GTK_CONTAINER(content), 20);

    top = gtk_box_new(GTK_ORIENTATION_VERTICAL, 6);
    title = gtk_label_new("  RAILWAY E-TICKET");
    gtk_style_context_add_class(gtk_widget_get_style_context(title), "ticket-title");
    gtk_box_pack_start(GTK_BOX(top), title, FALSE, FALSE, 0);

    status = gtk_label_new(is_waiting ? "WAITING LIST" : "CONFIRMED RESERVATION");
    gtk_style_context_add_class(gtk_widget_get_style_context(status),
                                is_waiting ? "status-waiting" : "status-confirmed");
    gtk_box_pack_start(GTK_BOX(top), status, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(content), top, FALSE, FALSE, 4);

    grid = gtk_grid_new();
    gtk_grid_set_row_spacing(GTK_GRID(grid), 8);
    gtk_grid_set_column_spacing(GTK_GRID(grid), 18);
    gtk_container_set_border_width(GTK_CONTAINER(grid), 16);
    gtk_style_context_add_class(gtk_widget_get_style_context(grid), "ticket-card");

#define ADD_TICKET_ROW(lbl, val) do { \
    GtkWidget *k = gtk_label_new(lbl); \
    GtkWidget *v = gtk_label_new(val); \
    gtk_label_set_xalign(GTK_LABEL(k), 0.0f); \
    gtk_label_set_xalign(GTK_LABEL(v), 0.0f); \
    gtk_style_context_add_class(gtk_widget_get_style_context(k), "ticket-key"); \
    gtk_style_context_add_class(gtk_widget_get_style_context(v), "ticket-value"); \
    gtk_grid_attach(GTK_GRID(grid), k, 0, ticket_row, 1, 1); \
    gtk_grid_attach(GTK_GRID(grid), v, 1, ticket_row, 1, 1); \
    ticket_row++; \
} while (0)

    {
        int ticket_row = 0;
        snprintf(text, sizeof text, "%d", p->ticket_id); ADD_TICKET_ROW("Ticket ID", text);
        ADD_TICKET_ROW("PNR", p->pnr);
        ADD_TICKET_ROW("Passenger", p->name);
        snprintf(text, sizeof text, "%d", p->age); ADD_TICKET_ROW("Age", text);
        ADD_TICKET_ROW("Gender", p->gender);
        ADD_TICKET_ROW("Phone", p->phone);
        ADD_TICKET_ROW("Email", p->email);
        ADD_TICKET_ROW("Class", class_name(p->travel_class));
        ADD_TICKET_ROW("Berth Preference", p->berth);
        ADD_TICKET_ROW("From", p->from);
        ADD_TICKET_ROW("To", p->to);
```

```
        ADD_TICKET_ROW("Journey Date", p->date);
        if (is_waiting) {
            snprintf(text, sizeof text, "WL %d", p->waiting_position);
            ADD_TICKET_ROW("Position", text);
        } else {
            ADD_TICKET_ROW("Seat", p->seat);
        }
    }
#undef ADD_TICKET_ROW

    gtk_box_pack_start(GTK_BOX(content), grid, FALSE, FALSE, 6);

    note = gtk_label_new("Keep your PNR or Ticket ID for quick search and cancellation.\nBerth preference
 is recorded but not guaranteed by this educational simulator.");
    gtk_label_set_line_wrap(GTK_LABEL(note), TRUE);
    gtk_style_context_add_class(gtk_widget_get_style_context(note), "ticket-note");
    gtk_box_pack_start(GTK_BOX(content), note, FALSE, FALSE, 8);

    copy_button = gtk_dialog_add_button(GTK_DIALOG(dialog), "Copy PNR", 1);
    save_button = gtk_dialog_add_button(GTK_DIALOG(dialog), "Save Ticket", 2);
    close_button = gtk_dialog_add_button(GTK_DIALOG(dialog), "Close", GTK_RESPONSE_CLOSE);
    gtk_style_context_add_class(gtk_widget_get_style_context(copy_button), "soft");
    gtk_style_context_add_class(gtk_widget_get_style_context(save_button), "primary");
    (void)close_button;

    gtk_widget_show_all(dialog);

    for (;;) {
        response = gtk_dialog_run(GTK_DIALOG(dialog));
        if (response == 1) {
            GtkClipboard *clipboard = gtk_clipboard_get(GDK_SELECTION_CLIPBOARD);
            gtk_clipboard_set_text(clipboard, p->pnr, -1);
            log_msg("PNR COPIED: %s", p->pnr);
            show_notification("PNR copied to clipboard.", GTK_MESSAGE_INFO);
        } else if (response == 2) {
            save_ticket_to_file(p, GTK_WINDOW(dialog));
        } else {
            break;
        }
    }

    gtk_widget_destroy(dialog);
}

```

12.8 Event handlers: booking, cancellation, search, reset (lines 1201 – 1635)

```
static void on_book(GtkWidget *widget, gpointer data)
{
    gchar *gender = NULL;
    gchar *class_code = NULL;
    gchar *berth = NULL;
    const char *name;
    const char *phone;
    const char *email;
    const char *date;
    int age;
    int class_cap;
    int class_confirmed;
    Passenger *p;
    char clean_name[NAME_LEN];
    char clean_phone[PHONE_LEN];
    char clean_email[EMAIL_LEN];
    char clean_date[DATE_LEN];

    (void)widget;
    (void)data;

    name = gtk_entry_get_text(GTK_ENTRY(entry_name));
    phone = gtk_entry_get_text(GTK_ENTRY(entry_phone));
    email = gtk_entry_get_text(GTK_ENTRY(entry_email));
    date = gtk_entry_get_text(GTK_ENTRY(entry_date));
    age = gtk_spin_button_get_value_as_int(GTK_SPIN_BUTTON(spin_age));
    gender = gtk_combo_box_text_get_active_text(GTK_COMBO_BOX_TEXT(combo_gender));
    class_code = gtk_combo_box_text_get_active_text(GTK_COMBO_BOX_TEXT(combo_class));
    berth = gtk_combo_box_text_get_active_text(GTK_COMBO_BOX_TEXT(combo_berth));

    copy_text(clean_name, sizeof clean_name, name);
```

```
    copy_text(clean_phone, sizeof clean_phone, phone);
    copy_text(clean_email, sizeof clean_email, email);
    copy_text(clean_date, sizeof clean_date, date);
    trim_spaces(clean_name);
    trim_spaces(clean_phone);
    trim_spaces(clean_email);
    trim_spaces(clean_date);

    if (!gender || !class_code || !berth) {
        log_msg("ERROR: complete the selection fields before booking.");
        show_notification("Please complete the booking selections.", GTK_MESSAGE_WARNING);
        goto cleanup;
    }

    if (clean_name[0] == '\0') {
        log_msg("ERROR: enter passenger name.");
        show_notification("Passenger name is required.", GTK_MESSAGE_WARNING);
        goto cleanup;
    }

    if (!valid_phone(clean_phone)) {
        log_msg("ERROR: phone number must contain 10–15 digits.");
        show_notification("Enter a valid phone number.", GTK_MESSAGE_WARNING);
        goto cleanup;
    }

    if (!valid_email(clean_email)) {
        log_msg("ERROR: enter a valid email address.");
        show_notification("Enter a valid email address.", GTK_MESSAGE_WARNING);
        goto cleanup;
    }

    if (!valid_date(clean_date)) {
        log_msg("ERROR: journey date must use DD-MM-YYYY.");
        show_notification("Use a valid DD-MM-YYYY journey date.", GTK_MESSAGE_WARNING);
        goto cleanup;
    }

    if (confirmed.size >= MAX_CONFIRMED && waiting.size >= MAX_WAITING) {
        log_msg("TRAIN FULL: confirmed and waiting lists are full.");
        show_notification("Train and waiting list are full.", GTK_MESSAGE_ERROR);
        goto cleanup;
    }

    class_cap = class_capacity(class_code);
    class_confirmed = confirmed_class_count(class_code);

    p = calloc(1, sizeof *p);
    if (!p) {
        log_msg("ERROR: out of memory while creating passenger.");
        show_notification("Could not create the reservation.", GTK_MESSAGE_ERROR);
        goto cleanup;
    }

    p->ticket_id = next_ticket++;
    make_pnr(p->pnr, sizeof p->pnr);
    p->age = age;
    copy_text(p->name, sizeof p->name, clean_name);
    copy_text(p->gender, sizeof p->gender, gender);
    copy_text(p->phone, sizeof p->phone, clean_phone);
    copy_text(p->email, sizeof p->email, clean_email);
    copy_text(p->travel_class, sizeof p->travel_class, class_code);
    copy_text(p->berth, sizeof p->berth, berth);
    copy_text(p->from, sizeof p->from, "Thiruvananthapuram");
    copy_text(p->to, sizeof p->to, "New Delhi");
    copy_text(p->date, sizeof p->date, clean_date);

    if (confirmed.size < MAX_CONFIRMED && class_confirmed < class_cap) {
        make_seat(p->seat, sizeof p->seat, class_code);
        enqueue(&confirmed, p);
        total_bookings++;

        log_msg("CONFIRMED: #%d | PNR %s | %s | %s | Seat %s",
                p->ticket_id, p->pnr, p->name, class_code, p->seat);
        show_notification("Booking confirmed — seat allocated.", GTK_MESSAGE_INFO);
    } else {
        if (waiting.size >= MAX_WAITING) {
            free(p);
            log_msg("ERROR: waiting list is full.");
            show_notification("Waiting list is full.", GTK_MESSAGE_ERROR);
```

```
            goto cleanup;
        }

        p->seat[0] = '\0';
        p->waiting_position = waiting.size + 1;
        enqueue(&waiting, p);
        total_bookings++;

        log_msg("WAITING: #%d | PNR %s | %s | %s | WL %d",
                p->ticket_id, p->pnr, p->name, class_code, p->waiting_position);
        show_notification("Class full — passenger added to FIFO waiting list.", GTK_MESSAGE_WARNING);
    }

    selected_ticket_id = p->ticket_id;
    clear_booking_fields();
    refresh_tables();
    refresh_details();
    show_ticket_dialog(p);

cleanup:
    g_free(gender);
    g_free(class_code);
    g_free(berth);
}

static int ask_confirmation(const char *title, const char *secondary, GtkMessageType type)
{
    GtkWidget *dialog;
    gint response;

    if (!confirm_actions)
        return 1;

    dialog = gtk_message_dialog_new(GTK_WINDOW(window),
                                    GTK_DIALOG_MODAL,
                                    type,
                                    GTK_BUTTONS_YES_NO,
                                    "%s", title);
    gtk_message_dialog_format_secondary_text(GTK_MESSAGE_DIALOG(dialog), "%s", secondary);
    response = gtk_dialog_run(GTK_DIALOG(dialog));
    gtk_widget_destroy(dialog);
    return response == GTK_RESPONSE_YES;
}

static void cancel_ticket_id(int ticket_id)
{
    Passenger *candidate;
    Passenger *gone;
    Passenger *promoted;
    char old_seat[SEAT_LEN];
    char class_code[CLASS_LEN];

    candidate = find_ticket(ticket_id);
    if (!candidate) {
        log_msg("ERROR: ticket #%d was not found.", ticket_id);
        show_notification("Ticket ID was not found.", GTK_MESSAGE_ERROR);
        return;
    }

    if (candidate->seat[0] != '\0') {
        if (!ask_confirmation("Cancel this confirmed ticket?",
                              "The released seat may be assigned immediately to the first waiting
passenger of the same class.",
                              GTK_MESSAGE_WARNING))
            return;

        gone = remove_ticket(&confirmed, ticket_id);
        if (!gone) {
            log_msg("ERROR: ticket #%d could not be removed safely.", ticket_id);
            show_notification("The ticket could not be cancelled.", GTK_MESSAGE_ERROR);
            return;
        }

        copy_text(old_seat, sizeof old_seat, gone->seat);
        copy_text(class_code, sizeof class_code, gone->travel_class);

        log_msg("CANCELLED: #%d | %s | Seat %s", gone->ticket_id, gone->name, old_seat);
        free(gone);
        total_cancelled++;

```

```
        promoted = remove_first_class(&waiting, class_code);
        if (promoted) {
            promoted->waiting_position = 0;
            copy_text(promoted->seat, sizeof promoted->seat, old_seat);
            enqueue(&confirmed, promoted);
            total_promoted++;

            log_msg("PROMOTED: #%d | %s | %s | Seat %s",
                    promoted->ticket_id, promoted->name,
                    promoted->travel_class, promoted->seat);
            show_notification("Waiting passenger automatically promoted.", GTK_MESSAGE_INFO);
        } else {
            log_msg("SEAT RELEASED: %s is now available.", old_seat);
            show_notification("Ticket cancelled and seat released.", GTK_MESSAGE_INFO);
        }
    } else {
        if (!ask_confirmation("Cancel this waiting-list ticket?",
                              "This waiting-list position will be removed and the remaining queue will move up.",
                              GTK_MESSAGE_WARNING))
            return;

        gone = remove_ticket(&waiting, ticket_id);
        if (!gone) {
            log_msg("ERROR: ticket #%d could not be removed safely.", ticket_id);
            show_notification("The waiting-list ticket could not be cancelled.", GTK_MESSAGE_ERROR);
            return;
        }

        log_msg("WAITING CANCELLED: #%d | %s | PNR %s",
                gone->ticket_id, gone->name, gone->pnr);
        free(gone);
        total_cancelled++;
        show_notification("Waiting-list ticket cancelled.", GTK_MESSAGE_INFO);
    }

    selected_ticket_id = -1;
    gtk_entry_set_text(GTK_ENTRY(entry_cancel), "");
    refresh_tables();
}

static void on_cancel(GtkWidget *widget, gpointer data)
{
    const char *text;
    int ticket_id;

    (void)widget;
    (void)data;

    text = gtk_entry_get_text(GTK_ENTRY(entry_cancel));
    if (!parse_ticket_id(text, &ticket_id)) {
        log_msg("ERROR: enter a valid numeric ticket ID.");
        show_notification("Enter a valid numeric ticket ID.", GTK_MESSAGE_WARNING);
        return;
    }

    cancel_ticket_id(ticket_id);
}

static void on_cancel_selected(GtkWidget *widget, gpointer data)
{
    Passenger *p;

    (void)widget;
    (void)data;

    p = find_ticket(selected_ticket_id);
    if (!p) {
        show_notification("Select a passenger first.", GTK_MESSAGE_WARNING);
        return;
    }

    cancel_ticket_id(p->ticket_id);
}

static void on_search(GtkWidget *widget, gpointer data)
{
    char clean[PNR_LEN];
    const char *query;
    int ticket_id;
```

```
    Passenger *p = NULL;

    (void)widget;
    (void)data;

    query = gtk_entry_get_text(GTK_ENTRY(entry_search));
    copy_text(clean, sizeof clean, query);
    trim_spaces(clean);

    if (clean[0] == '\0') {
        show_notification("Enter a PNR or Ticket ID to search.", GTK_MESSAGE_WARNING);
        return;
    }

    if (parse_ticket_id(clean, &ticket_id))
        p = find_ticket(ticket_id);
    if (!p)
        p = find_pnr(clean);

    if (!p) {
        log_msg("SEARCH: no ticket / PNR found for '%s'.", clean);
        show_notification("No matching ticket was found.", GTK_MESSAGE_ERROR);
        return;
    }

    selected_ticket_id = p->ticket_id;
    refresh_details();

    gtk_entry_set_text(GTK_ENTRY(entry_cancel), "");
    {
        char text[32];
        snprintf(text, sizeof text, "%d", p->ticket_id);
        gtk_entry_set_text(GTK_ENTRY(entry_cancel), text);
    }

    log_msg("SEARCH: FOUND #%d | %s | PNR %s", p->ticket_id, p->name, p->pnr);
    show_notification("Passenger found — details updated below.", GTK_MESSAGE_INFO);
}

static void on_confirmed_selected(GtkTreeSelection *selection, gpointer data)
{
    GtkTreeModel *model;
    GtkTreeIter iter;
    int ticket_id;
    char text[32];

    (void)data;

    if (!gtk_tree_selection_get_selected(selection, &model, &iter))
        return;

    gtk_tree_model_get(model, &iter, 0, &ticket_id, -1);
    selected_ticket_id = ticket_id;
    snprintf(text, sizeof text, "%d", ticket_id);
    gtk_entry_set_text(GTK_ENTRY(entry_cancel), text);
    refresh_details();
}

static void on_waiting_selected(GtkTreeSelection *selection, gpointer data)
{
    GtkTreeModel *model;
    GtkTreeIter iter;
    int ticket_id;
    char text[32];

    (void)data;

    if (!gtk_tree_selection_get_selected(selection, &model, &iter))
        return;

    gtk_tree_model_get(model, &iter, 1, &ticket_id, -1);
    selected_ticket_id = ticket_id;
    snprintf(text, sizeof text, "%d", ticket_id);
    gtk_entry_set_text(GTK_ENTRY(entry_cancel), text);
    refresh_details();
}

static void on_view_selected(GtkWidget *widget, gpointer data)
{
    Passenger *p;
```

```
1550
1551      (void)widget;
1552      (void)data;
1553
1554      p = find_ticket(selected_ticket_id);
1555      if (!p) {
1556          show_notification("Select a passenger first.", GTK_MESSAGE_WARNING);
1557          return;
1558      }
1559      show_ticket_dialog(p);
1560  }
1561
1562  static void on_filter_changed(GtkWidget *widget, gpointer data)
1563  {
1564      (void)widget;
1565      (void)data;
1566      refresh_tables();
1567  }
1568
1569  static void on_clear_log(GtkWidget *widget, gpointer data)
1570  {
1571      (void)widget;
1572      (void)data;
1573
1574      gtk_text_buffer_set_text(log_buffer, "", -1);
1575      gtk_text_buffer_set_text(dash_activity_buffer, "", -1);
1576      gtk_label_set_text(GTK_LABEL(lbl_status), "Activity log cleared.");
1577  }
1578
1579  static void clear_system_state(const char *log_text)
1580  {
1581      free_queue(&confirmed);
1582      free_queue(&waiting);
1583      queue_init(&confirmed);
1584      queue_init(&waiting);
1585
1586      next_ticket = 1001;
1587      next_pnr = 4817263951ULL;
1588      total_bookings = 0;
1589      total_cancelled = 0;
1590      total_promoted = 0;
1591      selected_ticket_id = -1;
1592
1593      gtk_entry_set_text(GTK_ENTRY(entry_name), "");
1594      gtk_entry_set_text(GTK_ENTRY(entry_phone), "");
1595      gtk_entry_set_text(GTK_ENTRY(entry_email), "");
1596      gtk_entry_set_text(GTK_ENTRY(entry_date), "");
1597      gtk_entry_set_text(GTK_ENTRY(entry_search), "");
1598      gtk_entry_set_text(GTK_ENTRY(entry_cancel), "");
1599
1600      gtk_text_buffer_set_text(log_buffer, "", -1);
1601      gtk_text_buffer_set_text(dash_activity_buffer, "", -1);
1602      refresh_tables();
1603      refresh_details();
1604      update_booking_availability();
1605      log_msg("%s", log_text);
1606  }
1607
1608  static void on_reset(GtkWidget *widget, gpointer data)
1609  {
1610      (void)widget;
1611      (void)data;
1612
1613      if (!ask_confirmation("Reset the complete reservation system?",
1614                            "All current bookings, counters, PNRs and waiting-list entries will be
      deleted.",
1615                            GTK_MESSAGE_WARNING))
1616          return;
1617
1618      clear_system_state("SYSTEM RESET: a completely fresh reservation journey is ready.");
1619      show_notification("Reservation system reset.", GTK_MESSAGE_INFO);
1620  }
1621
1622  static void on_new_journey(GtkWidget *widget, gpointer data)
1623  {
1624      (void)widget;
1625      (void)data;
1626
1627      if (!ask_confirmation("Start a new journey?",
```

```
1628                            "All current passenger records will be cleared and the booking counters will
    restart.",
1629                            GTK_MESSAGE_QUESTION))
1630          return;
1631
1632      clear_system_state("NEW JOURNEY: reservation system is ready for the next trip.");
1633      show_notification("New journey created.", GTK_MESSAGE_INFO);
1634  }
1635
```

12.9 Notifications, clock, theme and navigation (lines 1636 – 1789)

```
1636  /* ---------- Notifications ---------- */
1637
1638  static GtkWidget *notification_bar;
1639  static GtkWidget *notification_label;
1640  static guint notification_source = 0;
1641
1642  static gboolean hide_notification_cb(gpointer data)
1643  {
1644      (void)data;
1645      if (notification_bar)
1646          gtk_widget_hide(notification_bar);
1647      notification_source = 0;
1648      return G_SOURCE_REMOVE;
1649  }
1650
1651  static void show_notification(const char *message, GtkMessageType type)
1652  {
1653      if (!notification_bar || !notification_label) return;
1654
1655      if (notification_source != 0) {
1656          g_source_remove(notification_source);
1657          notification_source = 0;
1658      }
1659
1660      gtk_info_bar_set_message_type(GTK_INFO_BAR(notification_bar), type);
1661      gtk_label_set_text(GTK_LABEL(notification_label), message);
1662      gtk_widget_show(notification_bar);
1663      notification_source = g_timeout_add(3200, hide_notification_cb, NULL);
1664  }
1665
1666  /* ---------- Header clock ---------- */
1667
1668  static gboolean update_clock(gpointer data)
1669  {
1670      char text[64];
1671      time_t now;
1672      struct tm *tm_info;
1673
1674      (void)data;
1675
1676      now = time(NULL);
1677      tm_info = localtime(&now);
1678      if (!tm_info || !header_clock)
1679          return G_SOURCE_CONTINUE;
1680
1681      strftime(text, sizeof text, "%a %d %b  •  %I:%M:%S %p", tm_info);
1682      gtk_label_set_text(GTK_LABEL(header_clock), text);
1683      return G_SOURCE_CONTINUE;
1684  }
1685
1686  /* ---------- Theme ---------- */
1687
1688  static void apply_theme(void)
1689  {
1690      GtkStyleContext *ctx;
1691
1692      if (!window) return;
1693      ctx = gtk_widget_get_style_context(window);
1694
1695      if (dark_mode)
1696          gtk_style_context_add_class(ctx, "dark-mode");
1697      else
1698          gtk_style_context_remove_class(ctx, "dark-mode");
1699  }
1700
1701  static void on_dark_mode_changed(GtkSwitch *sw, GParamSpec *pspec, gpointer data)
```

```
{
    (void)pspec;
    (void)data;

    dark_mode = gtk_switch_get_active(sw);
    apply_theme();
    log_msg("SETTINGS: dark mode %s.", dark_mode ? "enabled" : "disabled");
}

static void on_confirm_setting_changed(GtkSwitch *sw, GParamSpec *pspec, gpointer data)
{
    (void)pspec;
    (void)data;

    confirm_actions = gtk_switch_get_active(sw);
    log_msg("SETTINGS: confirmation dialogs %s.", confirm_actions ? "enabled" : "disabled");
}

/* ---------- Navigation ---------- */

static void go_to_page(const char *name)
{
    if (main_stack && name)
        gtk_stack_set_visible_child_name(GTK_STACK(main_stack), name);
}

static void on_go_booking(GtkWidget *widget, gpointer data)
{
    (void)widget;
    (void)data;
    go_to_page("booking");
    gtk_widget_grab_focus(entry_name);
}

static void on_go_passengers(GtkWidget *widget, gpointer data)
{
    (void)widget;
    (void)data;
    go_to_page("passengers");
}

static void on_go_activity(GtkWidget *widget, gpointer data)
{
    (void)widget;
    (void)data;
    go_to_page("activity");
}

/* ---------- TreeView ---------- */

static GtkWidget *make_tree(GtkListStore *store,
                            const char **columns,
                            int count,
                            gboolean waiting_table)
{
    GtkWidget *tree;
    GtkTreeSelection *selection;
    int i;

    tree = gtk_tree_view_new_with_model(GTK_TREE_MODEL(store));
    gtk_tree_view_set_headers_visible(GTK_TREE_VIEW(tree), TRUE);
    gtk_tree_view_set_grid_lines(GTK_TREE_VIEW(tree), GTK_TREE_VIEW_GRID_LINES_HORIZONTAL);
    gtk_tree_view_set_enable_search(GTK_TREE_VIEW(tree), FALSE);
    gtk_tree_view_set_activate_on_single_click(GTK_TREE_VIEW(tree), FALSE);

    for (i = 0; i < count; i++) {
        GtkCellRenderer *renderer = gtk_cell_renderer_text_new();
        GtkTreeViewColumn *column =
            gtk_tree_view_column_new_with_attributes(columns[i], renderer, "text", i, NULL);

        gtk_tree_view_column_set_resizable(column, TRUE);
        gtk_tree_view_column_set_sort_column_id(column, i);
        if (i == 2 || i == 5)
            gtk_tree_view_column_set_expand(column, TRUE);
        gtk_tree_view_append_column(GTK_TREE_VIEW(tree), column);
    }

    selection = gtk_tree_view_get_selection(GTK_TREE_VIEW(tree));
    gtk_tree_selection_set_mode(selection, GTK_SELECTION_SINGLE);

```

```
    if (waiting_table)
        g_signal_connect(selection, "changed", G_CALLBACK(on_waiting_selected), NULL);
    else
        g_signal_connect(selection, "changed", G_CALLBACK(on_confirmed_selected), NULL);

    return tree;
}

```

12.10 TreeView setup and CSS styling (lines 1790 – 1953)

```
/* ---------- CSS ---------- */

static void load_css(void)
{
    GString *css_builder = g_string_new(NULL);
    g_string_append(css_builder, "* { font-family: Sans; }");
    g_string_append(css_builder, "window { background: #eef2f7; color: #18233f; }");
    g_string_append(css_builder, "headerbar { background: #10245b; color: #ffffff; min-height: 72px; border: none; }");
    g_string_append(css_builder, "headerbar .title { color: #ffffff; font-size: 20px; font-weight: 900; }");
    g_string_append(css_builder, "headerbar .subtitle { color: #b9c9f3; font-size: 10px; }");
    g_string_append(css_builder, ".header-box { padding: 4px 8px; }");
    g_string_append(css_builder, ".header-clock { color: #dce7ff; font-size: 11px; font-weight: 700; }");
    g_string_append(css_builder, ".online { color: #63e6a5; font-weight: 900; font-size: 11px; }");
    g_string_append(css_builder, ".sidebar { background: #0c1735; padding: 12px 8px; border-right: 1px solid #20335f; }");
    g_string_append(css_builder, ".sidebar row { color: #d5e0fa; border-radius: 10px; padding: 11px 9px; margin: 3px 0; font-weight: 700; }");
    g_string_append(css_builder, ".sidebar row:hover { background: #162d61; color: #ffffff; }");
    g_string_append(css_builder, ".sidebar row:selected { background: #2856c7; color: #ffffff; }");
    g_string_append(css_builder, ".sidebar row label { color: inherit; font-size: 13px; font-weight: 800; }");
    g_string_append(css_builder, ".page { padding: 16px; background: #eef2f7; }");
    g_string_append(css_builder, ".hero { background: #162d70; border-radius: 18px; }");
    g_string_append(css_builder, ".hero-box { padding: 18px 22px; }");
    g_string_append(css_builder, ".hero-title { color: #ffffff; font-size: 26px; font-weight: 900; }");
    g_string_append(css_builder, ".hero-sub { color: #d6e1ff; font-size: 11px; }");
    g_string_append(css_builder, ".route-city { color: #ffffff; font-size: 13px; font-weight: 900; }");
    g_string_append(css_builder, ".route-time { color: #bdd0ff; font-size: 11px; }");
    g_string_append(css_builder, ".route-line { color: #84a4ff; font-size: 18px; font-weight: 900; }");
    g_string_append(css_builder, ".stat-title { color: #68748a; font-size: 10px; font-weight: 900; }");
    g_string_append(css_builder, ".stat-icon { font-size: 15px; }");
    g_string_append(css_builder, ".stat-value { color: #18233f; font-size: 25px; font-weight: 900; }");
    g_string_append(css_builder, ".card { background: #ffffff; border: 1px solid #dbe3ef; border-radius: 14px; }");
    g_string_append(css_builder, ".card:hover { border-color: #b9c8e4; }");
    g_string_append(css_builder, ".accent-blue { border-top: 3px solid #3159c7; }");
    g_string_append(css_builder, ".accent-amber { border-top: 3px solid #e0a11b; }");
    g_string_append(css_builder, ".accent-green { border-top: 3px solid #1e9d68; }");
    g_string_append(css_builder, ".accent-red { border-top: 3px solid #cc4638; }");
    g_string_append(css_builder, ".section-title { color: #18233f; font-size: 16px; font-weight: 900; }");
    g_string_append(css_builder, ".section-subtitle { color: #7b879b; font-size: 10px; }");
    g_string_append(css_builder, ".mini { color: #77839a; font-size: 10px; }");
    g_string_append(css_builder, ".class-card { background: #ffffff; border: 1px solid #dbe3ef; border-radius: 13px; }");
    g_string_append(css_builder, ".class-code { color: #18233f; font-size: 14px; font-weight: 900; }");
    g_string_append(css_builder, ".class-free { color: #73809a; font-size: 10px; font-weight: 700; }");
    g_string_append(css_builder, "progressbar trough { min-height: 8px; border-radius: 6px; background: #e6ebf3; }");
    g_string_append(css_builder, "progressbar progress { min-height: 8px; border-radius: 6px; background: #3159c7; }");
    g_string_append(css_builder, ".dark-progress progress { background: #6d8df5; }");
    g_string_append(css_builder, ".quick-title { color: #6c7890; font-size: 10px; font-weight: 900; }");
    g_string_append(css_builder, "button { border-radius: 8px; padding: 8px 14px; font-weight: 700; }");
    g_string_append(css_builder, "button:hover { background: #e7edfb; }");
    g_string_append(css_builder, ".primary { background: #2856c7; color: #ffffff; border: 1px solid #2856c7; font-weight: 900; padding: 10px 18px; }");
    g_string_append(css_builder, ".primary:hover { background: #1f47aa; color: #ffffff; }");
    g_string_append(css_builder, ".danger { background: #b72d22; color: #ffffff; border: 1px solid #b72d22; font-weight: 900; }");
    g_string_append(css_builder, ".danger:hover { background: #92231b; color: #ffffff; }");
    g_string_append(css_builder, ".soft { background: #e9efff; color: #24459d; border: 1px solid #cedcff; }");
    g_string_append(css_builder, ".soft:hover { background: #dce6ff; }");
    g_string_append(css_builder, ".ghost { background: transparent; color: #3159c7; border: 1px solid #cbd7ee; }");
```

```
1844      g_string_append(css_builder, ".form-card { background: #ffffff; border: 1px solid #dbe3ef; border-
    radius: 14px; padding: 16px; }");
1845      g_string_append(css_builder, ".field-label { color: #536078; font-size: 10px; font-weight: 900; }");
1846      g_string_append(css_builder, "entry { min-height: 36px; border-radius: 8px; padding: 5px 9px; }");
1847      g_string_append(css_builder, "combobox box { min-height: 34px; }");
1848      g_string_append(css_builder, "spinbutton { min-height: 34px; }");
1849      g_string_append(css_builder, ".availability-card { background: #f5f8ff; border: 1px solid #d5e0fb;
    border-radius: 12px; padding: 11px; }");
1850      g_string_append(css_builder, ".success-text { color: #18794f; font-weight: 800; font-size: 11px; }");
1851      g_string_append(css_builder, ".warning-text { color: #a96100; font-weight: 800; font-size: 11px; }");
1852      g_string_append(css_builder, ".recent-list { background: transparent; }");
1853      g_string_append(css_builder, ".recent-list row { background: #ffffff; border: 1px solid #dbe3ef;
    border-radius: 10px; margin: 3px 0; }");
1854      g_string_append(css_builder, ".recent-list row:hover { background: #f4f7fd; }");
1855      g_string_append(css_builder, ".recent-ticket { color: #3159c7; font-size: 11px; font-weight: 900; min-
    width: 56px; }");
1856      g_string_append(css_builder, ".recent-name { color: #18233f; font-size: 12px; font-weight: 900; }");
1857      g_string_append(css_builder, ".empty-text { color: #8a95a8; font-size: 11px; padding: 22px; }");
1858      g_string_append(css_builder, ".badge { border-radius: 999px; padding: 3px 8px; font-size: 9px; font-
    weight: 900; }");
1859      g_string_append(css_builder, ".badge-confirmed { background: #dff6ea; color: #14754e; }");
1860      g_string_append(css_builder, ".badge-waiting { background: #fff0cf; color: #9b5b00; }");
1861      g_string_append(css_builder, ".table-card { background: #ffffff; border: 1px solid #dbe3ef; border-
    radius: 14px; padding: 10px; }");
1862      g_string_append(css_builder, "treeview { background: #ffffff; color: #1c2741; border: none; }");
1863      g_string_append(css_builder, "treeview header button { color: #46526a; font-weight: 900; background:
    #f5f7fb; }");
1864      g_string_append(css_builder, "treeview.view row:selected { background: #dce7ff; color: #17294f; }");
1865      g_string_append(css_builder, ".detail-card { background: #f8faff; border: 1px solid #d9e2f3; border-
    radius: 14px; padding: 14px; }");
1866      g_string_append(css_builder, ".detail-name { color: #18233f; font-size: 18px; font-weight: 900; }");
1867      g_string_append(css_builder, ".detail-key { color: #7a869a; font-size: 9px; font-weight: 900; }");
1868      g_string_append(css_builder, ".detail-value { color: #26324a; font-size: 11px; font-weight: 800; }");
1869      g_string_append(css_builder, ".status-confirmed { color: #16754d; font-weight: 900; }");
1870      g_string_append(css_builder, ".status-waiting { color: #a35d00; font-weight: 900; }");
1871      g_string_append(css_builder, ".activity-view { background: #0a1329; color: #dce7ff; padding: 10px;
    border-radius: 10px; }");
1872      g_string_append(css_builder, ".activity-title { color: #ffffff; font-size: 11px; font-weight: 900;
    }");
1873      g_string_append(css_builder, ".ticket-title { color: #152a6e; font-size: 21px; font-weight: 900; }");
1874      g_string_append(css_builder, ".ticket-card { background: #f7f9fd; border: 1px solid #dfe6f1; border-
    radius: 13px; padding: 12px; }");
1875      g_string_append(css_builder, ".ticket-key { color: #78849a; font-size: 10px; font-weight: 900; }");
1876      g_string_append(css_builder, ".ticket-value { color: #1e2a44; font-size: 11px; font-weight: 800; }");
1877      g_string_append(css_builder, ".ticket-note { color: #728098; font-size: 10px; }");
1878      g_string_append(css_builder, ".setting-card { background: #ffffff; border: 1px solid #dbe3ef; border-
    radius: 14px; padding: 15px; }");
1879      g_string_append(css_builder, ".setting-title { color: #18233f; font-size: 13px; font-weight: 900; }");
1880      g_string_append(css_builder, ".setting-desc { color: #7a869a; font-size: 10px; }");
1881      g_string_append(css_builder, ".footer { color: #7e8a9e; font-size: 9px; padding: 2px; }");
1882      g_string_append(css_builder, ".infobar { border-radius: 9px; margin: 8px 12px; }");
1883      g_string_append(css_builder, "window.dark-mode { background: #0c1220; color: #e7ecf8; }");
1884      g_string_append(css_builder, "window.dark-mode .page { background: #101726; }");
1885      g_string_append(css_builder, "window.dark-mode .card, window.dark-mode .form-card, window.dark-mode
    .table-card, window.dark-mode .detail-card, window.dark-mode .setting-card { background: #182235; border-
    color: #2a3852; }");
1886      g_string_append(css_builder, "window.dark-mode .section-title, window.dark-mode .detail-name,
    window.dark-mode .setting-title, window.dark-mode .stat-value, window.dark-mode .class-code, window.dark-
    mode .recent-name { color: #eef3ff; }");
1887      g_string_append(css_builder, "window.dark-mode .mini, window.dark-mode .section-subtitle, window.dark-
    mode .detail-key, window.dark-mode .field-label, window.dark-mode .setting-desc { color: #9eabc2; }");
1888      g_string_append(css_builder, "window.dark-mode entry, window.dark-mode spinbutton, window.dark-mode
    combobox box { background: #111a2c; color: #edf2ff; border-color: #33425f; }");
1889      g_string_append(css_builder, "window.dark-mode treeview { background: #182235; color: #e9efff; }");
1890      g_string_append(css_builder, "window.dark-mode treeview header button { background: #121a2b; color:
    #cdd8ee; }");
1891      g_string_append(css_builder, "window.dark-mode treeview.view row:selected { background: #294b8f;
    color: #ffffff; }");
1892      g_string_append(css_builder, "window.dark-mode .recent-list row { background: #182235; border-color:
    #2a3852; }");
1893      g_string_append(css_builder, "window.dark-mode .availability-card { background: #15223b; border-color:
    #31476f; }");
1894      g_string_append(css_builder, "window.dark-mode .detail-value { color: #dae3f5; }");
1895      g_string_append(css_builder, "window.dark-mode .ticket-card { background: #182235; border-color:
    #2a3852; }");
1896      g_string_append(css_builder, "window.dark-mode .ticket-key { color: #9eabc2; }");
1897      g_string_append(css_builder, "window.dark-mode .ticket-value { color: #edf2ff; }");
1898      g_string_append(css_builder, "window.dark-mode .ticket-title { color: #b8cbff; }");
1899      const char *css = css_builder->str;
1900
```

```
    GtkCssProvider *provider = gtk_css_provider_new();
    gtk_css_provider_load_from_data(provider, css, -1, NULL);
    gtk_style_context_add_provider_for_screen(
        gdk_screen_get_default(), GTK_STYLE_PROVIDER(provider),
        GTK_STYLE_PROVIDER_PRIORITY_APPLICATION);
    g_object_unref(provider);
    g_string_free(css_builder, TRUE);
}

/* ---------- Widget builders ---------- */

static GtkWidget *make_info_pair(const char *key, GtkWidget **value_out)
{
    GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 2);
    GtkWidget *k = make_label(key, "detail-key");
    GtkWidget *v = make_label("—", "detail-value");
    gtk_box_pack_start(GTK_BOX(box), k, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(box), v, FALSE, FALSE, 0);
    *value_out = v;
    return box;
}

static GtkWidget *make_route_card(void)
{
    GtkWidget *card = make_card();
    GtkWidget *box = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 12);
    GtkWidget *left = gtk_box_new(GTK_ORIENTATION_VERTICAL, 2);
    GtkWidget *center = gtk_box_new(GTK_ORIENTATION_VERTICAL, 1);
    GtkWidget *right = gtk_box_new(GTK_ORIENTATION_VERTICAL, 2);

    gtk_container_add(GTK_CONTAINER(card), box);
    gtk_container_set_border_width(GTK_CONTAINER(box), 12);

    gtk_box_pack_start(GTK_BOX(left), make_label("THIRUVANANTHAPURAM", "route-city"), FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(left), make_label("12:30 PM  •  Departure", "route-time"), FALSE, FALSE,
0);

    gtk_widget_set_hexpand(center, TRUE);
    {
        GtkWidget *route_line = make_label("● ———————— ◆ ———————— ●", "route-line");
        gtk_label_set_line_wrap(GTK_LABEL(route_line), TRUE);
        gtk_box_pack_start(GTK_BOX(center), route_line, FALSE, FALSE, 0);
    }
    gtk_box_pack_start(GTK_BOX(center), make_label("KERALA EXPRESS  •  12625", "mini"), FALSE, FALSE, 0);

    gtk_box_pack_start(GTK_BOX(right), make_label("NEW DELHI", "route-city"), FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(right), make_label("06:20 AM  •  Arrival", "route-time"), FALSE, FALSE, 0);

    gtk_box_pack_start(GTK_BOX(box), left, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(box), center, TRUE, TRUE, 0);
    gtk_box_pack_end(GTK_BOX(box), right, FALSE, FALSE, 0);
    return card;
}

```

12.11 Page builders: dashboard, booking, passengers, activity, settings (lines 1954 – 2574)

```
/* ---------- Window construction ---------- */

static void build_dashboard_page(GtkWidget *stack)
{
    GtkWidget *page = make_page();
    GtkWidget *hero;
    GtkWidget *hero_box;
    GtkWidget *title;
    GtkWidget *subtitle;
    GtkWidget *stats;
    GtkWidget *mini_stats;
    GtkWidget *class_row;
    GtkWidget *section_row;
    GtkWidget *recent_card;
    GtkWidget *recent_box;
    GtkWidget *recent_title;
    GtkWidget *recent_sw;
    GtkWidget *activity_card;
    GtkWidget *activity_box;
```

```
    GtkWidget *activity_title;
    GtkWidget *activity_sw;
    GtkWidget *quick;
    GtkWidget *quick_title;
    GtkWidget *quick_buttons;
    GtkWidget *button;
    GtkWidget *footer;

    hero = gtk_event_box_new();
    gtk_style_context_add_class(gtk_widget_get_style_context(hero), "hero");
    hero_box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 5);
    gtk_container_add(GTK_CONTAINER(hero), hero_box);
    gtk_container_set_border_width(GTK_CONTAINER(hero_box), 18);

    title = make_label("  KERALA EXPRESS", "hero-title");
    subtitle = make_label("Train 12625  •  Queue-based reservation management  •  System Online", "hero-sub");
    gtk_box_pack_start(GTK_BOX(hero_box), title, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(hero_box), subtitle, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(hero_box), make_route_card(), FALSE, FALSE, 5);
    gtk_box_pack_start(GTK_BOX(page), hero, FALSE, FALSE, 0);

    stats = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    gtk_box_set_homogeneous(GTK_BOX(stats), TRUE);
    gtk_box_pack_start(GTK_BOX(stats), make_stat_card("✓", "CONFIRMED", &lbl_confirmed, "accent-blue"), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(stats), make_stat_card("◌", "WAITING", &lbl_waiting, "accent-amber"), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(stats), make_stat_card("▣", "AVAILABLE", &lbl_available, "accent-green"), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(stats), make_stat_card("×", "CANCELLED", &lbl_cancelled, "accent-red"), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(page), stats, FALSE, FALSE, 0);

    mini_stats = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    gtk_box_set_homogeneous(GTK_BOX(mini_stats), TRUE);
    gtk_box_pack_start(GTK_BOX(mini_stats), make_stat_card("Σ", "TOTAL BOOKINGS", &dash_total, "accent-blue"), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(mini_stats), make_stat_card("↗", "AUTO PROMOTED", &dash_promoted, "accent-green"), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(page), mini_stats, FALSE, FALSE, 0);

    gtk_box_pack_start(GTK_BOX(page),
                       make_section_header("Coach / Class Availability",
                                           "Live occupancy across the configured 38-seat train capacity."),
                       FALSE, FALSE, 0);

    class_row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    gtk_box_set_homogeneous(GTK_BOX(class_row), TRUE);
    gtk_box_pack_start(GTK_BOX(class_row), make_class_card("SL", "Sleeper", 0), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(class_row), make_class_card("3A", "AC 3 Tier", 1), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(class_row), make_class_card("2A", "AC 2 Tier", 2), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(class_row), make_class_card("1A", "AC First", 3), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(page), class_row, FALSE, FALSE, 0);

    section_row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    gtk_box_set_homogeneous(GTK_BOX(section_row), TRUE);

    recent_card = make_card();
    recent_box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 7);
    gtk_container_add(GTK_CONTAINER(recent_card), recent_box);
    gtk_container_set_border_width(GTK_CONTAINER(recent_box), 12);
    recent_title = make_label("Recent Bookings", "section-title");
    gtk_box_pack_start(GTK_BOX(recent_box), recent_title, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(recent_box), make_label("Click a reservation to open the e-ticket.", "section-subtitle"), FALSE, FALSE, 0);
    recent_list = GTK_LIST_BOX(gtk_list_box_new());
    gtk_style_context_add_class(gtk_widget_get_style_context(GTK_WIDGET(recent_list)), "recent-list");
    gtk_list_box_set_selection_mode(recent_list, GTK_SELECTION_NONE);
    g_signal_connect(recent_list, "row-activated", G_CALLBACK(on_recent_activated), NULL);
    recent_sw = gtk_scrolled_window_new(NULL, NULL);
    gtk_scrolled_window_set_policy(GTK_SCROLLED_WINDOW(recent_sw), GTK_POLICY_NEVER, GTK_POLICY_AUTOMATIC);
    gtk_widget_set_vexpand(recent_sw, TRUE);
    gtk_widget_set_size_request(recent_sw, -1, 185);
    gtk_container_add(GTK_CONTAINER(recent_sw), GTK_WIDGET(recent_list));
    gtk_box_pack_start(GTK_BOX(recent_box), recent_sw, TRUE, TRUE, 0);

    activity_card = make_card();
```

```
    activity_box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 7);
    gtk_container_add(GTK_CONTAINER(activity_card), activity_box);
    gtk_container_set_border_width(GTK_CONTAINER(activity_box), 12);
    activity_title = make_label("Recent Activity", "section-title");
    gtk_box_pack_start(GTK_BOX(activity_box), activity_title, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(activity_box), make_label("Live queue events, reservations and promotions.", "section-subtitle"), FALSE, FALSE, 0);
    {
        GtkWidget *view = gtk_text_view_new();
        GtkWidget *scroll = gtk_scrolled_window_new(NULL, NULL);
        gtk_text_view_set_editable(GTK_TEXT_VIEW(view), FALSE);
        gtk_text_view_set_cursor_visible(GTK_TEXT_VIEW(view), FALSE);
        gtk_text_view_set_monospace(GTK_TEXT_VIEW(view), TRUE);
        gtk_widget_set_name(view, "dashboard-activity");
        gtk_style_context_add_class(gtk_widget_get_style_context(view), "activity-view");
        dash_activity_buffer = gtk_text_view_get_buffer(GTK_TEXT_VIEW(view));
        gtk_widget_set_vexpand(scroll, TRUE);
        gtk_widget_set_size_request(scroll, -1, 185);
        gtk_container_add(GTK_CONTAINER(scroll), view);
        gtk_box_pack_start(GTK_BOX(activity_box), scroll, TRUE, TRUE, 0);
        activity_sw = scroll;
    }
    (void)activity_sw;

    gtk_box_pack_start(GTK_BOX(section_row), recent_card, TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(section_row), activity_card, TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(page), section_row, TRUE, TRUE, 0);

    quick = make_card();
    {
        GtkWidget *qb = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
        gtk_container_add(GTK_CONTAINER(quick), qb);
        gtk_container_set_border_width(GTK_CONTAINER(qb), 10);
        quick_title = make_label("QUICK ACTIONS", "quick-title");
        quick_buttons = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 7);
        gtk_widget_set_hexpand(quick_buttons, TRUE);

        button = gtk_button_new_with_label("  New Booking");
        gtk_style_context_add_class(gtk_widget_get_style_context(button), "primary");
        g_signal_connect(button, "clicked", G_CALLBACK(on_go_booking), NULL);
        gtk_box_pack_start(GTK_BOX(quick_buttons), button, FALSE, FALSE, 0);

        button = gtk_button_new_with_label("  Manage Passengers");
        gtk_style_context_add_class(gtk_widget_get_style_context(button), "soft");
        g_signal_connect(button, "clicked", G_CALLBACK(on_go_passengers), NULL);
        gtk_box_pack_start(GTK_BOX(quick_buttons), button, FALSE, FALSE, 0);

        button = gtk_button_new_with_label("  Open Activity");
        gtk_style_context_add_class(gtk_widget_get_style_context(button), "soft");
        g_signal_connect(button, "clicked", G_CALLBACK(on_go_activity), NULL);
        gtk_box_pack_start(GTK_BOX(quick_buttons), button, FALSE, FALSE, 0);

        gtk_box_pack_start(GTK_BOX(qb), quick_title, FALSE, FALSE, 4);
        gtk_box_pack_start(GTK_BOX(qb), quick_buttons, TRUE, TRUE, 0);
    }
    gtk_box_pack_start(GTK_BOX(page), quick, FALSE, FALSE, 0);

    footer = make_label("FIFO queue engine  •  GTK3  •  Seat capacities: SL 20 / 3A 10 / 2A 5 / 1A 3  •  Waiting limit: 10", "footer");
    gtk_label_set_xalign(GTK_LABEL(footer), 0.5f);
    gtk_box_pack_start(GTK_BOX(page), footer, FALSE, FALSE, 0);

    gtk_stack_add_titled(GTK_STACK(stack), page, "dashboard", "  Dashboard");
}

static void build_booking_page(GtkWidget *stack)
{
    GtkWidget *page = make_page();
    GtkWidget *header;
    GtkWidget *content_row;
    GtkWidget *form_card;
    GtkWidget *journey_card;
    GtkWidget *grid;
    GtkWidget *book_button;
    GtkWidget *availability_card;
    GtkWidget *availability_box;
    GtkWidget *tip_card;
    GtkWidget *tip_box;
    int row = 0;

```

```
    header = make_section_header("Create a New Reservation",
                                 "Complete passenger and journey details. The queue engine decides
confirmed or waiting status automatically.");
    gtk_box_pack_start(GTK_BOX(page), header, FALSE, FALSE, 0);

    content_row = gtk_paned_new(GTK_ORIENTATION_HORIZONTAL);
    gtk_paned_set_wide_handle(GTK_PANED(content_row), TRUE);
    gtk_widget_set_hexpand(content_row, TRUE);
    gtk_widget_set_vexpand(content_row, FALSE);

    form_card = make_card();
    gtk_style_context_add_class(gtk_widget_get_style_context(form_card), "form-card");
    grid = gtk_grid_new();
    gtk_grid_set_row_spacing(GTK_GRID(grid), 10);
    gtk_grid_set_column_spacing(GTK_GRID(grid), 9);
    gtk_container_add(GTK_CONTAINER(form_card), grid);

    gtk_grid_attach(GTK_GRID(grid), make_field_label("Full Name"), 0, row, 1, 1);
    entry_name = gtk_entry_new();
    gtk_entry_set_placeholder_text(GTK_ENTRY(entry_name), "Passenger full name");
    gtk_widget_set_hexpand(entry_name, TRUE);
    gtk_grid_attach(GTK_GRID(grid), entry_name, 1, row, 3, 1);
    gtk_grid_attach(GTK_GRID(grid), make_field_label("Age"), 4, row, 1, 1);
    spin_age = gtk_spin_button_new_with_range(1, 120, 1);
    gtk_spin_button_set_value(GTK_SPIN_BUTTON(spin_age), 25);
    gtk_grid_attach(GTK_GRID(grid), spin_age, 5, row, 1, 1);
    row++;

    gtk_grid_attach(GTK_GRID(grid), make_field_label("Gender"), 0, row, 1, 1);
    combo_gender = gtk_combo_box_text_new();
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_gender), "Male");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_gender), "Female");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_gender), "Other");
    gtk_combo_box_set_active(GTK_COMBO_BOX(combo_gender), 0);
    gtk_grid_attach(GTK_GRID(grid), combo_gender, 1, row, 1, 1);
    gtk_grid_attach(GTK_GRID(grid), make_field_label("Phone"), 2, row, 1, 1);
    entry_phone = gtk_entry_new();
    gtk_entry_set_placeholder_text(GTK_ENTRY(entry_phone), "10–15 digit phone");
    gtk_grid_attach(GTK_GRID(grid), entry_phone, 3, row, 3, 1);
    row++;

    gtk_grid_attach(GTK_GRID(grid), make_field_label("Email"), 0, row, 1, 1);
    entry_email = gtk_entry_new();
    gtk_entry_set_placeholder_text(GTK_ENTRY(entry_email), "Passenger email address");
    gtk_grid_attach(GTK_GRID(grid), entry_email, 1, row, 5, 1);
    row++;

    gtk_grid_attach(GTK_GRID(grid), make_field_label("Class"), 0, row, 1, 1);
    combo_class = gtk_combo_box_text_new();
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_class), "SL");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_class), "3A");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_class), "2A");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_class), "1A");
    gtk_combo_box_set_active(GTK_COMBO_BOX(combo_class), 0);
    gtk_grid_attach(GTK_GRID(grid), combo_class, 1, row, 1, 1);

    gtk_grid_attach(GTK_GRID(grid), make_field_label("Berth Preference"), 2, row, 1, 1);
    combo_berth = gtk_combo_box_text_new();
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_berth), "No Preference");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_berth), "Lower");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_berth), "Middle");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_berth), "Upper");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_berth), "Side Lower");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_berth), "Side Upper");
    gtk_combo_box_set_active(GTK_COMBO_BOX(combo_berth), 0);
    gtk_grid_attach(GTK_GRID(grid), combo_berth, 3, row, 3, 1);
    row++;

    gtk_grid_attach(GTK_GRID(grid), make_field_label("Journey Date"), 0, row, 1, 1);
    entry_date = gtk_entry_new();
    gtk_entry_set_placeholder_text(GTK_ENTRY(entry_date), "DD-MM-YYYY");
    gtk_entry_set_max_length(GTK_ENTRY(entry_date), 10);
    gtk_grid_attach(GTK_GRID(grid), entry_date, 1, row, 1, 1);
    book_button = gtk_button_new_with_label("  CONFIRM BOOKING");
    gtk_style_context_add_class(gtk_widget_get_style_context(book_button), "primary");
    gtk_widget_set_hexpand(book_button, TRUE);
    gtk_grid_attach(GTK_GRID(grid), book_button, 2, row, 4, 1);
    row++;

    availability_card = make_card();
```

```
    gtk_style_context_add_class(gtk_widget_get_style_context(availability_card), "availability-card");
    availability_box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 6);
    gtk_container_add(GTK_CONTAINER(availability_card), availability_box);
    gtk_container_set_border_width(GTK_CONTAINER(availability_box), 10);
    booking_availability_label = make_label("", "success-text");
    booking_availability_progress = gtk_progress_bar_new();
    gtk_box_pack_start(GTK_BOX(availability_box), make_label("LIVE CLASS AVAILABILITY", "quick-title"), FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(availability_box), booking_availability_label, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(availability_box), booking_availability_progress, FALSE, FALSE, 0);
    gtk_grid_attach(GTK_GRID(grid), availability_card, 0, row, 6, 1);

    gtk_paned_pack1(GTK_PANED(content_row), form_card, TRUE, FALSE);

    journey_card = make_card();
    {
        GtkWidget *jb = gtk_box_new(GTK_ORIENTATION_VERTICAL, 10);
        GtkWidget *route = make_route_card();
        GtkWidget *line1 = make_label("Train information", "section-title");
        GtkWidget *line2 = make_label("Kerala Express (12625)", "setting-title");
        GtkWidget *line3 = make_label("Departure 12:30 PM  •  Arrival 06:20 AM", "mini");
        GtkWidget *line4 = make_label("Route: Thiruvananthapuram → New Delhi", "mini");
        GtkWidget *line5 = make_label("Configured capacity", "section-title");
        GtkWidget *line6 = make_label("SL 20  •  3A 10  •  2A 5  •  1A 3  •  Waiting 10", "mini");
        GtkWidget *line7 = make_label("Booking logic", "section-title");
        GtkWidget *line8 = make_label("Confirmed seats are assigned first. When a class is full, new passengers enter the FIFO waiting queue.", "mini");
        gtk_label_set_line_wrap(GTK_LABEL(line8), TRUE);
        gtk_label_set_line_wrap(GTK_LABEL(line4), TRUE);

        gtk_container_add(GTK_CONTAINER(journey_card), jb);
        gtk_container_set_border_width(GTK_CONTAINER(jb), 12);
        gtk_box_pack_start(GTK_BOX(jb), line1, FALSE, FALSE, 0);
        gtk_box_pack_start(GTK_BOX(jb), line2, FALSE, FALSE, 0);
        gtk_box_pack_start(GTK_BOX(jb), line3, FALSE, FALSE, 0);
        gtk_box_pack_start(GTK_BOX(jb), line4, FALSE, FALSE, 0);
        gtk_box_pack_start(GTK_BOX(jb), route, FALSE, FALSE, 3);
        gtk_box_pack_start(GTK_BOX(jb), line5, FALSE, FALSE, 2);
        gtk_box_pack_start(GTK_BOX(jb), line6, FALSE, FALSE, 0);
        gtk_box_pack_start(GTK_BOX(jb), line7, FALSE, FALSE, 2);
        gtk_box_pack_start(GTK_BOX(jb), line8, FALSE, FALSE, 0);
    }
    gtk_paned_pack2(GTK_PANED(content_row), journey_card, TRUE, FALSE);
    gtk_paned_set_position(GTK_PANED(content_row), 560);
    gtk_box_pack_start(GTK_BOX(page), content_row, FALSE, FALSE, 0);

    tip_card = make_card();
    tip_box = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    gtk_container_add(GTK_CONTAINER(tip_card), tip_box);
    gtk_container_set_border_width(GTK_CONTAINER(tip_box), 10);
    gtk_box_pack_start(GTK_BOX(tip_box), make_label("TIP", "badge"), FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(tip_box), make_label("After booking, an e-ticket opens automatically. You can copy the PNR or save the ticket as a text file.", "mini"), TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(page), tip_card, FALSE, FALSE, 0);

    g_signal_connect(book_button, "clicked", G_CALLBACK(on_book), NULL);
    g_signal_connect(entry_name, "activate", G_CALLBACK(on_book), NULL);
    g_signal_connect(combo_class, "changed", G_CALLBACK(update_booking_availability), NULL);

    gtk_stack_add_titled(GTK_STACK(stack), page, "booking", "  Book Ticket");
}

static void build_passenger_page(GtkWidget *stack)
{
    GtkWidget *page = make_page();
    GtkWidget *header;
    GtkWidget *search_card;
    GtkWidget *search_box;
    GtkWidget *search_button;
    GtkWidget *cancel_button;
    GtkWidget *filters;
    GtkWidget *filter_label;
    GtkWidget *tables_row;
    GtkWidget *conf_card;
    GtkWidget *wait_card;
    GtkWidget *conf_sw;
    GtkWidget *wait_sw;
    GtkWidget *conf_tree;
    GtkWidget *wait_tree;
    GtkWidget *details_card;
```

```
    GtkWidget *details_box;
    GtkWidget *details_top;
    GtkWidget *details_grid;
    GtkWidget *view_button;
    GtkWidget *cancel_selected_button;
    const char *conf_cols[] = {"Ticket", "PNR", "Passenger", "Class", "Seat", "Date", "Status"};
    const char *wait_cols[] = {"WL", "Ticket", "PNR", "Passenger", "Class", "Date", "Status"};

    header = make_section_header("Passenger Management",
                                 "Search, filter, inspect and cancel current reservations from one place.");
    gtk_box_pack_start(GTK_BOX(page), header, FALSE, FALSE, 0);

    search_card = make_card();
    search_box = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
    gtk_container_add(GTK_CONTAINER(search_card), search_box);
    gtk_container_set_border_width(GTK_CONTAINER(search_box), 10);

    entry_search = gtk_entry_new();
    gtk_entry_set_placeholder_text(GTK_ENTRY(entry_search), "Search by PNR or Ticket ID...");
    gtk_widget_set_hexpand(entry_search, TRUE);
    search_button = gtk_button_new_with_label("  Search");
    gtk_style_context_add_class(gtk_widget_get_style_context(search_button), "soft");

    entry_cancel = gtk_entry_new();
    gtk_entry_set_placeholder_text(GTK_ENTRY(entry_cancel), "Ticket ID to cancel");
    gtk_widget_set_hexpand(entry_cancel, TRUE);
    cancel_button = gtk_button_new_with_label("×  Cancel Ticket");
    gtk_style_context_add_class(gtk_widget_get_style_context(cancel_button), "danger");

    gtk_box_pack_start(GTK_BOX(search_box), entry_search, TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(search_box), search_button, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(search_box), gtk_separator_new(GTK_ORIENTATION_VERTICAL), FALSE, FALSE, 2);
    gtk_box_pack_start(GTK_BOX(search_box), entry_cancel, TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(search_box), cancel_button, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(page), search_card, FALSE, FALSE, 0);

    filters = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
    filter_label = make_label("FILTERS", "quick-title");
    combo_filter_class = gtk_combo_box_text_new();
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_class), "All Classes");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_class), "SL");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_class), "3A");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_class), "2A");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_class), "1A");
    gtk_combo_box_set_active(GTK_COMBO_BOX(combo_filter_class), 0);

    combo_filter_status = gtk_combo_box_text_new();
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_status), "All Status");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_status), "Confirmed");
    gtk_combo_box_text_append_text(GTK_COMBO_BOX_TEXT(combo_filter_status), "Waiting");
    gtk_combo_box_set_active(GTK_COMBO_BOX(combo_filter_status), 0);

    gtk_box_pack_start(GTK_BOX(filters), filter_label, FALSE, FALSE, 4);
    gtk_box_pack_start(GTK_BOX(filters), combo_filter_class, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(filters), combo_filter_status, FALSE, FALSE, 0);
    gtk_box_pack_end(GTK_BOX(filters), make_label("Click a row to load the selected passenger details below.", "mini"), FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(page), filters, FALSE, FALSE, 0);

    store_confirmed = gtk_list_store_new(7,
                                         G_TYPE_INT, G_TYPE_STRING, G_TYPE_STRING,
                                         G_TYPE_STRING, G_TYPE_STRING, G_TYPE_STRING,
                                         G_TYPE_STRING);
    store_waiting = gtk_list_store_new(7,
                                       G_TYPE_INT, G_TYPE_INT, G_TYPE_STRING,
                                       G_TYPE_STRING, G_TYPE_STRING, G_TYPE_STRING,
                                       G_TYPE_STRING);

    tables_row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
    gtk_box_set_homogeneous(GTK_BOX(tables_row), TRUE);

    conf_card = make_card();
    {
        GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 5);
        gtk_container_add(GTK_CONTAINER(conf_card), box);
        gtk_container_set_border_width(GTK_CONTAINER(box), 9);
        gtk_box_pack_start(GTK_BOX(box), make_label("CONFIRMED PASSENGERS", "section-title"), FALSE, FALSE, 0);
        conf_sw = gtk_scrolled_window_new(NULL, NULL);
```

```
2354          gtk_widget_set_vexpand(conf_sw, TRUE);
2355          conf_tree = make_tree(store_confirmed, conf_cols, 7, FALSE);
2356          gtk_container_add(GTK_CONTAINER(conf_sw), conf_tree);
2357          gtk_box_pack_start(GTK_BOX(box), conf_sw, TRUE, TRUE, 0);
2358      }
2359
2360      wait_card = make_card();
2361      {
2362          GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 5);
2363          gtk_container_add(GTK_CONTAINER(wait_card), box);
2364          gtk_container_set_border_width(GTK_CONTAINER(box), 9);
2365          gtk_box_pack_start(GTK_BOX(box), make_label("FIFO WAITING LIST", "section-title"), FALSE, FALSE,
      0);
2366          wait_sw = gtk_scrolled_window_new(NULL, NULL);
2367          gtk_widget_set_vexpand(wait_sw, TRUE);
2368          wait_tree = make_tree(store_waiting, wait_cols, 7, TRUE);
2369          gtk_container_add(GTK_CONTAINER(wait_sw), wait_tree);
2370          gtk_box_pack_start(GTK_BOX(box), wait_sw, TRUE, TRUE, 0);
2371      }
2372
2373      gtk_box_pack_start(GTK_BOX(tables_row), conf_card, TRUE, TRUE, 0);
2374      gtk_box_pack_start(GTK_BOX(tables_row), wait_card, TRUE, TRUE, 0);
2375      gtk_widget_set_vexpand(tables_row, TRUE);
2376      gtk_box_pack_start(GTK_BOX(page), tables_row, TRUE, TRUE, 0);
2377
2378      details_card = make_card();
2379      gtk_style_context_add_class(gtk_widget_get_style_context(details_card), "detail-card");
2380      details_box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 7);
2381      gtk_container_add(GTK_CONTAINER(details_card), details_box);
2382      gtk_container_set_border_width(GTK_CONTAINER(details_box), 10);
2383
2384      details_top = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
2385      details_grid = gtk_grid_new();
2386      gtk_grid_set_row_spacing(GTK_GRID(details_grid), 6);
2387      gtk_grid_set_column_spacing(GTK_GRID(details_grid), 18);
2388      gtk_widget_set_hexpand(details_grid, TRUE);
2389      gtk_box_pack_start(GTK_BOX(details_top), details_grid, TRUE, TRUE, 0);
2390
2391      {
2392          GtkWidget *name_label = make_label("Selected Passenger", "detail-name");
2393          gtk_grid_attach(GTK_GRID(details_grid), name_label, 0, 0, 2, 1);
2394          detail_name = name_label;
2395      }
2396      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Ticket ID", &detail_ticket), 0, 1, 1, 1);
2397      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("PNR", &detail_pnr), 1, 1, 1, 1);
2398      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Class", &detail_class), 2, 1, 1, 1);
2399      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Seat / Queue", &detail_seat), 3, 1, 1, 1);
2400      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Status", &detail_status), 4, 1, 1, 1);
2401      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Journey Date", &detail_date), 0, 2, 1, 1);
2402      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Phone", &detail_phone), 1, 2, 1, 1);
2403      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Email", &detail_email), 2, 2, 1, 1);
2404      gtk_grid_attach(GTK_GRID(details_grid), make_info_pair("Berth", &detail_berth), 3, 2, 1, 1);
2405
2406      view_button = gtk_button_new_with_label("  View E-Ticket");
2407      gtk_style_context_add_class(gtk_widget_get_style_context(view_button), "primary");
2408      cancel_selected_button = gtk_button_new_with_label("  Cancel Selected");
2409      gtk_style_context_add_class(gtk_widget_get_style_context(cancel_selected_button), "danger");
2410      gtk_box_pack_end(GTK_BOX(details_top), cancel_selected_button, FALSE, FALSE, 0);
2411      gtk_box_pack_end(GTK_BOX(details_top), view_button, FALSE, FALSE, 0);
2412      gtk_box_pack_start(GTK_BOX(details_box), details_top, FALSE, FALSE, 0);
2413
2414      gtk_box_pack_start(GTK_BOX(page), details_card, FALSE, FALSE, 0);
2415
2416      g_signal_connect(search_button, "clicked", G_CALLBACK(on_search), NULL);
2417      g_signal_connect(cancel_button, "clicked", G_CALLBACK(on_cancel), NULL);
2418      g_signal_connect(entry_search, "activate", G_CALLBACK(on_search), NULL);
2419      g_signal_connect(entry_cancel, "activate", G_CALLBACK(on_cancel), NULL);
2420      g_signal_connect(combo_filter_class, "changed", G_CALLBACK(on_filter_changed), NULL);
2421      g_signal_connect(combo_filter_status, "changed", G_CALLBACK(on_filter_changed), NULL);
2422      g_signal_connect(view_button, "clicked", G_CALLBACK(on_view_selected), NULL);
2423      g_signal_connect(cancel_selected_button, "clicked", G_CALLBACK(on_cancel_selected), NULL);
2424
2425      gtk_stack_add_titled(GTK_STACK(stack), page, "passengers", "  Passengers");
2426  }
2427
2428  static void build_activity_page(GtkWidget *stack)
2429  {
2430      GtkWidget *page = make_page();
2431      GtkWidget *header;
2432      GtkWidget *log_card;
```

```
    GtkWidget *box;
    GtkWidget *head;
    GtkWidget *clear_button;
    GtkWidget *scroll;
    GtkWidget *bottom;
    GtkWidget *new_button;
    GtkWidget *reset_button;
    GtkWidget *engine_card;

    header = make_section_header("System Activity",
                                 "Live audit trail of bookings, cancellations, queue promotions and system actions.");
    gtk_box_pack_start(GTK_BOX(page), header, FALSE, FALSE, 0);

    log_card = make_card();
    box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 8);
    gtk_container_add(GTK_CONTAINER(log_card), box);
    gtk_container_set_border_width(GTK_CONTAINER(box), 12);

    head = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
    lbl_status = make_label("System ready.", "activity-title");
    clear_button = gtk_button_new_with_label("Clear Log");
    gtk_style_context_add_class(gtk_widget_get_style_context(clear_button), "soft");
    gtk_box_pack_start(GTK_BOX(head), lbl_status, TRUE, TRUE, 0);
    gtk_box_pack_end(GTK_BOX(head), clear_button, FALSE, FALSE, 0);

    log_view = gtk_text_view_new();
    gtk_text_view_set_editable(GTK_TEXT_VIEW(log_view), FALSE);
    gtk_text_view_set_cursor_visible(GTK_TEXT_VIEW(log_view), FALSE);
    gtk_text_view_set_monospace(GTK_TEXT_VIEW(log_view), TRUE);
    gtk_style_context_add_class(gtk_widget_get_style_context(log_view), "activity-view");
    log_buffer = gtk_text_view_get_buffer(GTK_TEXT_VIEW(log_view));

    scroll = gtk_scrolled_window_new(NULL, NULL);
    gtk_widget_set_vexpand(scroll, TRUE);
    gtk_widget_set_size_request(scroll, -1, 390);
    gtk_container_add(GTK_CONTAINER(scroll), log_view);
    gtk_box_pack_start(GTK_BOX(box), head, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(box), scroll, TRUE, TRUE, 0);
    gtk_box_pack_start(GTK_BOX(page), log_card, TRUE, TRUE, 0);

    engine_card = make_card();
    {
        GtkWidget *ec = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 10);
        gtk_container_add(GTK_CONTAINER(engine_card), ec);
        gtk_container_set_border_width(GTK_CONTAINER(ec), 10);
        gtk_box_pack_start(GTK_BOX(ec), make_badge("FIFO", "badge-confirmed"), FALSE, FALSE, 0);
        gtk_box_pack_start(GTK_BOX(ec), make_label("Queue engine is active. Confirmed seats are allocated automatically; cancellations trigger same-class FIFO promotion.", "mini"), TRUE, TRUE, 0);
    }
    gtk_box_pack_start(GTK_BOX(page), engine_card, FALSE, FALSE, 0);

    bottom = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
    new_button = gtk_button_new_with_label("＋  New Journey");
    reset_button = gtk_button_new_with_label("↻  Reset System");
    gtk_style_context_add_class(gtk_widget_get_style_context(new_button), "soft");
    gtk_style_context_add_class(gtk_widget_get_style_context(reset_button), "danger");
    gtk_box_pack_end(GTK_BOX(bottom), reset_button, FALSE, FALSE, 0);
    gtk_box_pack_end(GTK_BOX(bottom), new_button, FALSE, FALSE, 0);
    gtk_box_pack_start(GTK_BOX(page), bottom, FALSE, FALSE, 0);

    g_signal_connect(clear_button, "clicked", G_CALLBACK(on_clear_log), NULL);
    g_signal_connect(new_button, "clicked", G_CALLBACK(on_new_journey), NULL);
    g_signal_connect(reset_button, "clicked", G_CALLBACK(on_reset), NULL);

    gtk_stack_add_titled(GTK_STACK(stack), page, "activity", "  Activity Log");
}

static void build_settings_page(GtkWidget *stack)
{
    GtkWidget *page = make_page();
    GtkWidget *header;
    GtkWidget *row;
    GtkWidget *train_card;
    GtkWidget *prefs_card;
    GtkWidget *about_card;

    header = make_section_header("Settings & System Info",
                                 "Application preferences and the configured educational train profile.");
    gtk_box_pack_start(GTK_BOX(page), header, FALSE, FALSE, 0);
```

```
2511
2512        row = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 12);
2513        gtk_box_set_homogeneous(GTK_BOX(row), TRUE);
2514
2515        train_card = make_card();
2516        {
2517            GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 8);
2518            gtk_container_add(GTK_CONTAINER(train_card), box);
2519            gtk_container_set_border_width(GTK_CONTAINER(box), 14);
2520            gtk_box_pack_start(GTK_BOX(box), make_label("TRAIN PROFILE", "section-title"), FALSE, FALSE, 0);
2521            gtk_box_pack_start(GTK_BOX(box), make_label("Kerala Express", "setting-title"), FALSE, FALSE, 0);
2522            gtk_box_pack_start(GTK_BOX(box), make_label("Train No. 12625", "mini"), FALSE, FALSE, 0);
2523            gtk_box_pack_start(GTK_BOX(box), make_label("Thiruvananthapuram → New Delhi", "mini"), FALSE,
    FALSE, 0);
2524            gtk_box_pack_start(GTK_BOX(box), make_label("Departure 12:30 PM  •  Arrival 06:20 AM", "mini"),
    FALSE, FALSE, 0);
2525            gtk_box_pack_start(GTK_BOX(box), make_label("Capacity: 38 confirmed + 10 waiting", "mini"), FALSE,
    FALSE, 0);
2526        }
2527
2528        prefs_card = make_card();
2529        {
2530            GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 12);
2531            GtkWidget *line;
2532            gtk_container_add(GTK_CONTAINER(prefs_card), box);
2533            gtk_container_set_border_width(GTK_CONTAINER(box), 14);
2534            gtk_box_pack_start(GTK_BOX(box), make_label("PREFERENCES", "section-title"), FALSE, FALSE, 0);
2535
2536            line = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
2537            gtk_box_pack_start(GTK_BOX(line), make_label("Dark Mode", "setting-title"), TRUE, TRUE, 0);
2538            switch_dark = gtk_switch_new();
2539            gtk_switch_set_active(GTK_SWITCH(switch_dark), dark_mode);
2540            gtk_box_pack_end(GTK_BOX(line), switch_dark, FALSE, FALSE, 0);
2541            gtk_box_pack_start(GTK_BOX(box), line, FALSE, FALSE, 0);
2542            gtk_box_pack_start(GTK_BOX(box), make_label("Switch between the polished light and dark dashboard
    themes.", "setting-desc"), FALSE, FALSE, 0);
2543
2544            line = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 8);
2545            gtk_box_pack_start(GTK_BOX(line), make_label("Confirmation Dialogs", "setting-title"), TRUE, TRUE,
    0);
2546            switch_confirm = gtk_switch_new();
2547            gtk_switch_set_active(GTK_SWITCH(switch_confirm), confirm_actions);
2548            gtk_box_pack_end(GTK_BOX(line), switch_confirm, FALSE, FALSE, 0);
2549            gtk_box_pack_start(GTK_BOX(box), line, FALSE, FALSE, 0);
2550            gtk_box_pack_start(GTK_BOX(box), make_label("Ask before cancellations, resets and new journeys.",
    "setting-desc"), FALSE, FALSE, 0);
2551        }
2552
2553        gtk_box_pack_start(GTK_BOX(row), train_card, TRUE, TRUE, 0);
2554        gtk_box_pack_start(GTK_BOX(row), prefs_card, TRUE, TRUE, 0);
2555        gtk_box_pack_start(GTK_BOX(page), row, FALSE, FALSE, 0);
2556
2557        about_card = make_card();
2558        {
2559            GtkWidget *box = gtk_box_new(GTK_ORIENTATION_VERTICAL, 8);
2560            gtk_container_add(GTK_CONTAINER(about_card), box);
2561            gtk_container_set_border_width(GTK_CONTAINER(box), 14);
2562            gtk_box_pack_start(GTK_BOX(box), make_label("ABOUT THIS MASTERPIECE", "section-title"), FALSE,
    FALSE, 0);
2563            gtk_box_pack_start(GTK_BOX(box), make_label("Railway Reservation System • Master V4", "setting-
    title"), FALSE, FALSE, 0);
2564            gtk_box_pack_start(GTK_BOX(box), make_label("C language + GTK3 + GLib. Designed around linked-list
    FIFO queues, automatic seat allocation, searchable passenger records, live statistics and an educational
    e-ticket workflow.", "setting-desc"), FALSE, FALSE, 0);
2565            gtk_box_pack_start(GTK_BOX(box), make_label("No external GUI toolkit is required beyond GTK3,
    keeping the build simple on Ubuntu.", "setting-desc"), FALSE, FALSE, 0);
2566        }
2567        gtk_box_pack_start(GTK_BOX(page), about_card, FALSE, FALSE, 0);
2568
2569        g_signal_connect(switch_dark, "notify::active", G_CALLBACK(on_dark_mode_changed), NULL);
2570        g_signal_connect(switch_confirm, "notify::active", G_CALLBACK(on_confirm_setting_changed), NULL);
2571
2572        gtk_stack_add_titled(GTK_STACK(stack), page, "settings", "⚙  Settings");
2573    }
2574
```

12.12 Main window and main() (lines 2575 – 2688)

```
2575 static void build_main_window(GtkApplication *app)
2576 {
2577     GtkWidget *root;
2578     GtkWidget *content;
2579     GtkWidget *sidebar;
2580     GtkWidget *header;
2581     GtkWidget *header_right;
2582     GtkWidget *stack_scroll;
2583 
2584     load_css();
2585 
2586     window = gtk_application_window_new(app);
2587     gtk_window_set_title(GTK_WINDOW(window), "Railway Reservation System — Master V4");
2588     gtk_window_set_default_size(GTK_WINDOW(window), 1120, 700);
2589     gtk_window_set_resizable(GTK_WINDOW(window), TRUE);
2590     gtk_window_set_position(GTK_WINDOW(window), GTK_WIN_POS_CENTER);
2591 
2592     header = gtk_header_bar_new();
2593     gtk_header_bar_set_show_close_button(GTK_HEADER_BAR(header), TRUE);
2594     gtk_header_bar_set_title(GTK_HEADER_BAR(header), "Railway Reservation");
2595     gtk_header_bar_set_subtitle(GTK_HEADER_BAR(header), "Kerala Express  •  12625  •  Thiruvananthapuram → New Delhi");
2596 
2597     header_right = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 12);
2598     gtk_style_context_add_class(gtk_widget_get_style_context(header_right), "header-box");
2599     header_status = make_label("● SYSTEM ONLINE", "online");
2600     header_clock = make_label("--:--:--", "header-clock");
2601     gtk_box_pack_start(GTK_BOX(header_right), header_status, FALSE, FALSE, 0);
2602     gtk_box_pack_start(GTK_BOX(header_right), header_clock, FALSE, FALSE, 0);
2603     gtk_header_bar_pack_end(GTK_HEADER_BAR(header), header_right);
2604     gtk_window_set_titlebar(GTK_WINDOW(window), header);
2605 
2606     root = gtk_box_new(GTK_ORIENTATION_VERTICAL, 0);
2607     gtk_container_add(GTK_CONTAINER(window), root);
2608 
2609     notification_bar = gtk_info_bar_new();
2610     gtk_style_context_add_class(gtk_widget_get_style_context(notification_bar), "infobar");
2611     {
2612         GtkWidget *area = gtk_info_bar_get_content_area(GTK_INFO_BAR(notification_bar));
2613         notification_label = gtk_label_new("");
2614         gtk_label_set_xalign(GTK_LABEL(notification_label), 0.0f);
2615         gtk_box_pack_start(GTK_BOX(area), notification_label, TRUE, TRUE, 0);
2616     }
2617     gtk_box_pack_start(GTK_BOX(root), notification_bar, FALSE, FALSE, 0);
2618 
2619     content = gtk_box_new(GTK_ORIENTATION_HORIZONTAL, 0);
2620     gtk_widget_set_vexpand(content, TRUE);
2621     gtk_box_pack_start(GTK_BOX(root), content, TRUE, TRUE, 0);
2622 
2623     main_stack = gtk_stack_new();
2624     gtk_stack_set_transition_type(GTK_STACK(main_stack), GTK_STACK_TRANSITION_TYPE_SLIDE_LEFT_RIGHT);
2625     gtk_stack_set_transition_duration(GTK_STACK(main_stack), 180);
2626     gtk_widget_set_hexpand(main_stack, TRUE);
2627     gtk_widget_set_vexpand(main_stack, TRUE);
2628 
2629     stack_scroll = gtk_scrolled_window_new(NULL, NULL);
2630     gtk_scrolled_window_set_policy(GTK_SCROLLED_WINDOW(stack_scroll),
2631                                    GTK_POLICY_NEVER, GTK_POLICY_AUTOMATIC);
2632     gtk_scrolled_window_set_overlay_scrolling(GTK_SCROLLED_WINDOW(stack_scroll), TRUE);
2633     gtk_widget_set_hexpand(stack_scroll, TRUE);
2634     gtk_widget_set_vexpand(stack_scroll, TRUE);
2635     gtk_container_add(GTK_CONTAINER(stack_scroll), main_stack);
2636 
2637     sidebar = gtk_stack_sidebar_new();
2638     gtk_style_context_add_class(gtk_widget_get_style_context(sidebar), "sidebar");
2639     gtk_widget_set_size_request(sidebar, 175, -1);
2640     gtk_stack_sidebar_set_stack(GTK_STACK_SIDEBAR(sidebar), GTK_STACK(main_stack));
2641 
2642     gtk_box_pack_start(GTK_BOX(content), sidebar, FALSE, FALSE, 0);
2643     gtk_box_pack_start(GTK_BOX(content), stack_scroll, TRUE, TRUE, 0);
2644 
2645     build_dashboard_page(main_stack);
2646     build_booking_page(main_stack);
2647     build_passenger_page(main_stack);
2648     build_activity_page(main_stack);
2649     build_settings_page(main_stack);
2650 
2651     gtk_stack_set_visible_child_name(GTK_STACK(main_stack), "dashboard");
```

```
    gtk_widget_show_all(window);
    /* gtk_widget_show_all() re-shows the hidden InfoBar, so hide it after the full tree is shown. */
    gtk_widget_hide(notification_bar);

    update_clock(NULL);
    g_timeout_add_seconds(1, update_clock, NULL);
    refresh_tables();
    update_booking_availability();
    log_msg("SYSTEM READY: Railway Reservation Master V4 started.");
    log_msg("FIFO queue engine active. Dashboard, booking, passenger and activity modules loaded.");
    log_msg("Seat capacity: SL20 / 3A10 / 2A5 / 1A3. Waiting limit: 10.");
}

/* ---------- Main ---------- */

int main(int argc, char **argv)
{
    GtkApplication *app;
    int status;

    queue_init(&confirmed);
    queue_init(&waiting);

    app = gtk_application_new("edu.railway.reservation.master.v4",
                              G_APPLICATION_DEFAULT_FLAGS);
    g_signal_connect(app, "activate", G_CALLBACK(build_main_window), NULL);

    status = g_application_run(G_APPLICATION(app), argc, argv);

    if (notification_source != 0)
        g_source_remove(notification_source);

    g_object_unref(app);
    free_queue(&confirmed);
    free_queue(&waiting);
    return status;
}
```

13. Advantages, Limitations and Future Scope

13.1 Advantages

- Pure pointer-based implementation, so every queue operation can be traced in the source.
- Fair, deterministic FIFO promotion within each class.
- Robust input handling and no fixed-size arrays for passenger storage.
- Clear separation between data structure, business rules and interface.
- Builds without warnings and needs only GTK3.

13.2 Limitations

- Data is held in memory only and is lost when the program closes.
- Single train with a fixed route and fixed capacities.
- Cancellation looks passengers up by scanning the list rather than by hashing.
- Single-user, single-machine operation.

13.3 Future scope

- Persist bookings to a file or SQLite database so records survive restarts.
- Use a hash table for O(1) Ticket ID and PNR lookup.
- Support several trains, dates and routes, with one pair of queues per journey.
- Add priority (senior citizen, ladies quota) using a priority queue.
- Add a network client-server mode for multiple booking counters.
- Export tickets as PDF and send them by e-mail.

14. Team Members and Contribution

No.	Name	Roll No.	Area of contribution
1	ANIL VK	10	Queue engine: Passenger and Queue structures, enqueue/dequeue, removal and promotion logic
2	AMAN AP	8	Booking module: validation, seat allocation, PNR and Ticket ID generation
3	MIDHUN M	25	GTK3 interface: dashboard, pages, styling, dark mode and notifications
4	WILLIAM WILSON	36	Testing, e-ticket, search and filters, documentation and report

15. Conclusion

The Railway Reservation System shows that a simple data structure, the linked-list queue, can drive a realistic application when it is applied with care. Two queues model confirmed and waiting passengers, FIFO order gives fairness, and the promotion rule ties the queues together in a way that exercises insertion at the rear, deletion at the front, and deletion from the middle. Around this core, validation, safe memory handling, searchable records and a clean graphical interface turn a classroom exercise into a usable tool.

The project strengthened our understanding of pointers, dynamic memory management, modular design and event-driven programming, and it leaves clear directions for extension such as persistence, hashing and multi-train support.

References

1. Kernighan, B. W. and Ritchie, D. M., The C Programming Language, 2nd ed., Prentice Hall.
2. Horowitz, E., Sahni, S. and Anderson-Freed, S., Fundamentals of Data Structures in C, 2nd ed., Universities Press.
3. Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C., Introduction to Algorithms, MIT Press.
4. GTK 3 Reference Manual, https://docs.gtk.org/gtk3/
5. GLib Reference Manual, https://docs.gtk.org/glib/
6. GCC Online Documentation, https://gcc.gnu.org/onlinedocs/